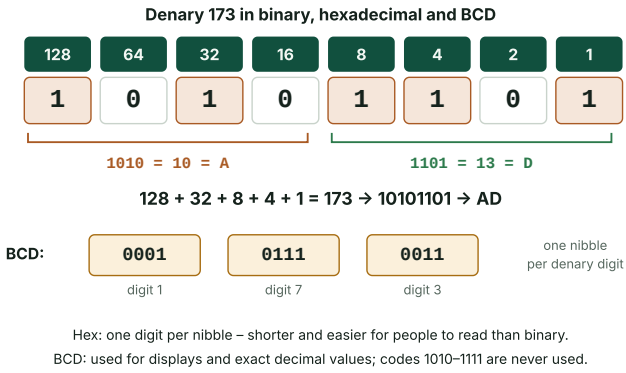
Number systems: bits, bytes, binary, denary, hexadecimal and BCD

FD01 FACT FILE

Specification: students should be able to

- Explain the terms bit, nibble, byte, kilobyte, megabyte, gigabyte and terabyte
- Demonstrate that 2^n different values can be represented with n bits
- Convert unsigned integers between denary and binary (up to 8 bits), showing working
- Convert between binary, denary and hexadecimal and explain why hexadecimal is used
- Describe binary-coded decimal (BCD) and its uses

Key words: bit, nibble, byte, kilobyte (KB), megabyte (MB), gigabyte (GB), terabyte (TB), place value, base 2 (binary), base 10 (denary), base 16 (hexadecimal), most significant bit (MSB), least significant bit (LSB), leading zeros, binary-coded decimal (BCD), colour code



173 converted to binary using place values, split into nibbles to give hexadecimal AD, and stored digit by digit in binary-coded decimal

Bits, bytes and number bases

Everything a computer stores – numbers, text, pictures, sound and the program instructions themselves – is held as patterns of bits, because the transistors and storage cells inside the machine have two reliable states. A single bit holds a 0 or a 1; four bits make a nibble and eight bits make a byte. A number base tells you how many different digits are available and what each column is worth. Denary (base 10) uses the digits 0 to 9 with column values that are powers of 10. Binary (base 2) uses only 0 and 1, with column values that double each time: 1, 2, 4, 8 and so on up to 128 in a byte. Hexadecimal (base 16) uses 0 to 9 followed by A to F for the values 10 to 15, with column values 1, 16, 256 and so on. The leftmost bit of a binary number has the highest place value and is called the most significant bit (MSB); the rightmost is the least significant bit (LSB). The same bit pattern can mean different things depending on the representation being used, which is why the rest of this topic is about agreed ways of interpreting bits.

Units of storage

Memory, disk capacity and file sizes are measured in bytes, and larger units are formed by multiplying by 1024, which is 2^{10}. A kilobyte is 1024 bytes, a megabyte is 1024 kilobytes (2^{20} bytes), a gigabyte is 1024 megabytes (2^{30} bytes) and a terabyte is 1024 gigabytes (2^{40} bytes). Each unit is therefore roughly a thousand times the one below, so a megabyte is about a million bytes and a gigabyte is about a billion. To convert to a larger unit divide by 1024; to convert to a smaller one multiply by 1024; to change bits into bytes divide by 8. Be aware that hard drive manufacturers and the international standards body use powers of 1000 for KB, MB and GB, and use the names kibibyte (KiB), mebibyte (MiB) and so on for the powers of 1024. This is why a new '1 TB' drive appears smaller when the operating system reports it. For CCEA answers, use the 1024-based definitions unless a question tells you otherwise, and always state the unit with your final answer.

Counting with n bits

Each extra bit doubles the number of different patterns available, because every existing pattern can be followed by either a 0 or a 1. One bit gives 2 patterns, two bits give 4, three bits give 8, and in general n bits give 2^n different values. As unsigned integers these run from 0 to $2^n - 1$, so a byte stores 0 to 255 and a 10-bit field stores 0 to 1023. The reverse question – how many bits are needed? – is answered by finding the smallest power of 2 that is at least the number of values required. To store every whole number from 0 to 1000 you need 1001 different values; 2^9 is only 512, but 2^{10} is 1024, so 10 bits are needed. Watch the off-by-one trap: storing 0 to 16 777 215 needs 16 777 216 values, which is exactly 2^{24}, so 24 bits are enough. Leading zeros are normally kept when numbers are stored or transmitted in a fixed width, because the receiving system must know exactly where one byte ends and the next begins; 1101 is sent as 00001101.

Converting between binary and denary

To convert binary to denary, write the place values above the bits, multiply each bit by its place value and add the results; only the columns containing 1 contribute. For example, 10011101 is 128 + 16 + 8 + 4 + 1 = 157. There are two reliable methods for converting denary to binary. In the subtraction method you write out the place values, place a 1 under the largest one that is not greater than the number, subtract it and repeat with the remainder until nothing is left, then fill the other columns with 0. In the division method you divide by 2 repeatedly, noting each remainder, until the quotient reaches 0; reading the remainders from the last to the first gives the binary number. Both methods give the same answer, and either earns the method mark if the working is shown. Always check your answer by converting back, and make sure the number of digits matches what the question asks for: '8-bit binary' means exactly eight digits, so add leading zeros where needed. A useful quick check is that every odd number ends in 1 and every even number ends in 0.

Hexadecimal

Hexadecimal is base 16, so it needs sixteen symbols: 0 to 9, then A, B, C, D, E and F for ten to fifteen. Because 16 is 2^4, one hex digit corresponds exactly to one nibble, and one byte is always two hex digits. To convert binary to hex, split the bits into groups of four starting from the right and replace each group with its hex digit; to convert back, expand each digit into four bits. To convert hex to denary, multiply the left digit by 16 and add the right digit, and for denary to hex divide by 16, using the quotient as the first digit and the remainder as the second. Hex is used because it is much shorter than binary, so people can read, write and remember values quickly and make fewer transcription errors, and it converts to binary far more simply than denary does. Typical uses are colour codes in HTML and CSS, MAC addresses, memory addresses shown in debugging tools, error codes and dumps of machine code. Hexadecimal is only a human-friendly notation: inside the computer the same values are still stored and processed as binary, so hex does not save any memory.

Binary-coded decimal

Binary-coded decimal stores each denary digit separately as its own 4-bit code, so 2026 is stored as 0010 0000 0010 0110 rather than as the pure binary value of the whole number. Only the ten patterns 0000 to 1001 are valid; the six patterns 1010 to 1111 are never used. The main advantage is that each nibble can be sent straight to a display digit, which is why BCD is found in digital clocks, calculators, seven-segment displays, meters and timers. It also allows decimal quantities, especially money, to be held exactly, avoiding the rounding errors that occur when values such as 0.1 are converted into binary fractions. The drawbacks are that BCD wastes storage, because six of the sixteen nibble patterns are unused (99 needs 8 bits in BCD but only 7 in pure binary), and arithmetic is more complex, because a result that produces an invalid nibble has to be corrected before it is a valid BCD digit again. A good answer compares BCD with pure binary on these points rather than simply describing it.

T Key terms from the mark schemes

Bit
A binary digit; the smallest unit of data, which stores either 0 or 1.
`Credited: binary digit`

Byte
A group of 8 bits, which can represent 256 different values.

Nibble
A group of 4 bits; half a byte, represented by one hexadecimal digit.

Kilobyte (KB)
1024 (2^{10}) bytes.
`Credited: approximately one thousand bytes`

Megabyte (MB)
1024 kilobytes, or 2^{20} (1 048 576) bytes.

Gigabyte (GB)
1024 megabytes, or 2^{30} bytes.

Terabyte (TB)
1024 gigabytes, or 2^{40} bytes.

Place value
The value of a column in a number, determined by its position and the base, e.g. 1, 2, 4, 8 in binary.
Credited: column weighting

Most significant bit (MSB)
The leftmost bit of a binary number, which has the highest place value.

Least significant bit (LSB)
The rightmost bit of a binary number, with a place value of 1.

Hexadecimal
A base-16 number system using the digits 0–9 and A–F, where each digit represents one nibble (4 bits).
Credited: base 16

Binary-coded decimal (BCD)
A representation in which each denary digit is stored separately as its own 4-bit binary code.

C C# example

Denary to binary: division method

```
int n = 214;
string bits = "";
while (n > 0)
{
    bits = (n % 2) + bits;   // remainder becomes the next bit on the left
    n = n / 2;               // integer division
}
bits = bits.PadLeft(8, '0');  // add leading zeros to make a full byte
Console.WriteLine(bits);      // 11010110
Console.WriteLine(Convert.ToInt32(bits, 2));  // back to 214
```

Exam focus

Conversion questions usually give one mark for method and one for the answer, so always show the place values, the subtractions, the remainders or the nibble split before the final value. Give binary answers in the number of bits asked for, adding leading zeros. For 'how many bits are needed', show the power of 2 you compared against, and remember that 0 counts as a value. When defining units, 'a kilobyte is 1024 bytes' or '2^{10} bytes' is credited; quoting 1000 bytes on its own may not be. 'Explain why hexadecimal is used' typically carries two marks per point, so give a reason and expand it, for example 'it is shorter than binary, so technicians make fewer errors when copying values'. Never claim that hex uses less memory or that the computer stores data in hex – both are marked wrong. For BCD, examiners want the idea that each denary digit is coded separately in four bits, plus a use such as a digital clock. Common mistakes are reading division remainders in the wrong order, forgetting that A = 10 rather than 11, and splitting binary into nibbles from the left instead of the right.

? Check yourself

1. State how many different values can be represented using 6 bits. [1]
2. A sports club in Omagh stores membership numbers from 0 to 1000 as unsigned integers. State the minimum number of bits needed for each membership number and justify your answer. [2]
3. Convert the 8-bit binary number 10011101 to denary. Show your working. [2]

Answers: 1. $2^6 = 64$ (1) 2. 10 bits (1) / 0 to 1000 is 1001 values; $2^9 = 512$ is not enough but $2^{10} = 1024$ is (1) 3. Place values / weightings shown: 128 + 16 + 8 + 4 + 1 (1) / = 157 (1)

Negative numbers and binary arithmetic

FD02 FACT FILE

Specification: students should be able to

- Perform binary addition of unsigned integers, showing carries
- Represent signed integers using sign and magnitude and explain its limitations
- Demonstrate how the two's complement system represents positive and negative numbers using 8 bits
- Convert between denary and 8-bit two's complement in both directions
- Perform addition and subtraction using two's complement
- Explain overflow and recognise when it occurs

Key words: binary addition, carry, signed integer, unsigned integer, sign bit, sign and magnitude, two's complement, invert (one's complement), negative place value, range, subtraction, overflow, status register (overflow flag), most significant bit (MSB)

Two's complement: the MSB is worth −128

	−128	64	32	16	8	4	2	1
+19	0	0	0	1	0	0	1	1
Invert every bit	1	1	1	0	1	1	0	0
Add 1 → −19	1	1	1	0	1	1	0	1

Check: −128 + 64 + 32 + 8 + 4 + 1 = −19 · Range in 8 bits: −128 to +127

Overflow

```
  01110000 (112)   Two positives gave a negative:
+ 00100000 ( 32)   the true answer 144 is
= 10010000 (reads as −112)   too big for +127, so overflow.
```

Forming -19 in 8-bit two's complement by inverting the bits of +19 and adding 1, and an example of overflow when two positive numbers give a negative result

Signed numbers in a fixed number of bits

Computers often need negative values – temperatures, bank balances, altitudes below sea level, changes in stock – but a binary register cannot hold a minus sign. Instead, one of the bits must carry the sign information and the whole system must agree how to interpret the pattern. Two schemes are examined. Sign and magnitude simply reserves the most significant bit as a flag. Two's complement gives the most significant bit a negative place value, which turns out to make arithmetic much simpler for hardware. Either way, the number of bits is fixed, usually eight in exam questions, and that fixes the range of values that can be stored. The same eight bits can be read in different ways: 10111001 is 185 as an unsigned integer, −57 in sign and magnitude and −71 in two's complement. This is why a question always tells you which representation to use, and why the answer must state it. Arithmetic on fixed-width numbers also introduces the risk of overflow, where the true answer lies outside the range. Understanding these ideas explains why programming languages offer data types such as byte, sbyte, int and long.

Binary addition

Binary addition follows the same column method as denary addition, but a carry happens as soon as a column total reaches two. The four rules are: 0 + 0 = 0; 0 + 1 = 1; 1 + 1 = 0 carry 1; and 1 + 1 + 1 = 1 carry 1. Work from the least significant bit on the right towards the left, writing any carry under the next column so it is included in that column's total. For example, adding 01011011 and 00110110 produces carries in six columns and gives 10010001, which is 145 – the same as 91 + 54. Always check the answer by converting each value to denary. Adding 1 to a run of 1s makes the carry ripple through every bit, which is why 0111 + 1 becomes 1000. When the numbers are unsigned, a carry out of the most significant bit means the result needs more bits than are available, which is overflow. When the numbers are in two's complement, the same circuit is used but the rule for detecting overflow changes, as explained below.

Sign and magnitude

In sign and magnitude the most significant bit is used only to show the sign, 0 for positive and 1 for negative, and the remaining bits give the size of the number in ordinary binary. So +19 is 00010011 and −19 is 10010011. It is easy for people to read, because the magnitude bits are identical for the positive and negative versions. However, it has two serious weaknesses. First, there are two representations of zero, 00000000 and 10000000, which wastes a pattern and means a comparison with zero must check two values. Second, and more importantly, ordinary binary addition gives wrong answers when the signs differ: 5 + (−3) worked as 00000101 + 10000011 gives 10001000, which reads as −8 rather than +2. A processor would need extra logic to compare signs and magnitudes and choose whether to add or subtract. Because one bit is used for the sign, n bits give a range of $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$, which is −127 to +127 for a byte.

Two's complement

In two's complement the most significant bit has a negative place value; in eight bits the columns are −128, 64, 32, 16, 8, 4, 2 and 1. Positive numbers look exactly as they do in unsigned binary and always start with 0; any pattern starting with 1 is negative. To convert a negative denary number, write the positive value in binary, pad it with leading zeros to eight bits, invert every bit and then add 1. To convert from two's complement to denary, add up the place values of the 1s, remembering that the first column is −128. There is only one zero, and the range for n bits is -2^{n-1} to $+(2^{n-1} - 1)$, so a byte holds −128 to +127. A useful check is that a number added to its two's complement gives eight zeros once the carry out is ignored. The great advantage is that subtraction becomes addition: A − B is calculated as A plus the two's complement of B, so the same adder circuit handles both operations and no sign-comparison logic is needed. This is why virtually all modern processors use two's complement for integers.

Subtraction and the discarded carry

To subtract in two's complement, first form the negative of the number being subtracted, then add. For 45 − 19, the two's complement of 19 is 11101101, and 00101101 + 11101101 gives a ninth bit: 1 00011010. Because the register only holds eight bits, the carry out of the most significant bit is discarded, leaving 00011010, which is 26, the correct answer. When the answer is negative, as in 38 − 52, there is no carry out: 00100110 + 11001100 gives 11110010, whose leading 1 tells you it is negative, and the place values give −128 + 64 + 32 + 16 + 2 = −14. Pupils often think the discarded carry is an error, but in two's complement arithmetic it is a normal part of the process whenever a positive and a negative number are added and the result is positive. It does not, by itself, indicate overflow. The correctness of the eight-bit result is decided by the signs of the inputs and output, not by whether a carry left the register.

Overflow

Overflow occurs when the true result of a calculation cannot be represented in the number of bits available. The test depends on the representation. For unsigned numbers, a carry out of the most significant bit means overflow: 200 + 100 = 300, which needs nine bits, so the eight-bit result 00101100 reads as 44. For two's complement, overflow happens only when two numbers with the same sign are added and the result has the opposite sign. Adding 100 and 50 gives 10010110, which reads as −106 because +150 is beyond +127; adding −100 and −50 gives 01101010, which reads as +106 because −150 is below −128. Adding a positive and a negative number can never overflow, since the result always lies between them. Processors record overflow in a flag in the status register, and well-written programs check it or use a larger data type such as a 32-bit int. If overflow goes undetected, the program carries on with a wrong value, which can produce incorrect output, a crash or, in control systems, dangerous behaviour.

T Key terms from the mark schemes

Binary addition
Adding binary numbers column by column from the right using the rules 0+0=0, 0+1=1, 1+1=0 carry 1, 1+1+1=1 carry 1.

Carry
A bit passed to the next column on the left when a column total is two or more.

Sign bit
The most significant bit of a signed number, used to show whether the value is positive or negative.

Sign and magnitude
A signed-integer representation in which the MSB indicates the sign (0 positive, 1 negative) and the remaining bits give the size.
`Credited: sign-magnitude`

Two's complement
A signed-integer representation in which the MSB has a negative place value; a number is negated by inverting its bits and adding 1.

Range (integers)
The smallest and largest values a representation can store with a given number of bits, e.g. −128 to +127 in 8-bit two's complement.

Overflow flag
A bit in the processor's status register that is set when an arithmetic operation overflows.

Unsigned integer
A whole number stored with no sign bit, so all bits represent magnitude and only zero or positive values are possible.

Overflow
An error that occurs when the result of a calculation is too large or too small to be stored in the available number of bits.

C C# example

Two's complement to denary

```
byte raw = 0b1011_1001;               // the stored 8 bits
sbyte value = unchecked((sbyte)raw);   // read them as two's complement
Console.WriteLine(raw);                // 185 if read as unsigned
Console.WriteLine(value);              // -71 if read as signed
```

Exam focus

Two's complement conversions usually carry a mark for the method and a mark for the answer, so show the positive binary padded to eight bits, the inverted bits and the result of adding 1, or show the place-value sum including −128. Check your answer by converting back; this also catches the common slip of forgetting to add 1. In addition questions, write the carry bits clearly beneath the sum and convert the result to denary when asked. For subtraction, state that you are adding the two's complement of the second number and say that the carry out of the most significant bit is discarded. When asked whether overflow has occurred, answer yes or no and justify it with numbers, for example 'the true answer is 150, which is greater than the largest value, 127, so overflow has occurred'. State ranges with both ends, such as −128 to +127, never −128 to +128. 'Why is two's complement preferred?' needs reasons such as a single zero and the ability to subtract using an adder, each briefly explained.

? Check yourself

1. State the range of integers that can be represented in 8-bit two's complement. [1]
2. Represent −45 in 8-bit sign and magnitude. [1]
3. Represent −45 in 8-bit two's complement. Show your working. [2]

Answers: 1. −128 to +127 (1) 2. 10101101 (1) 3. 45 = 00101101, invert to 11010010 (1) / add 1 = 11010011 (1)

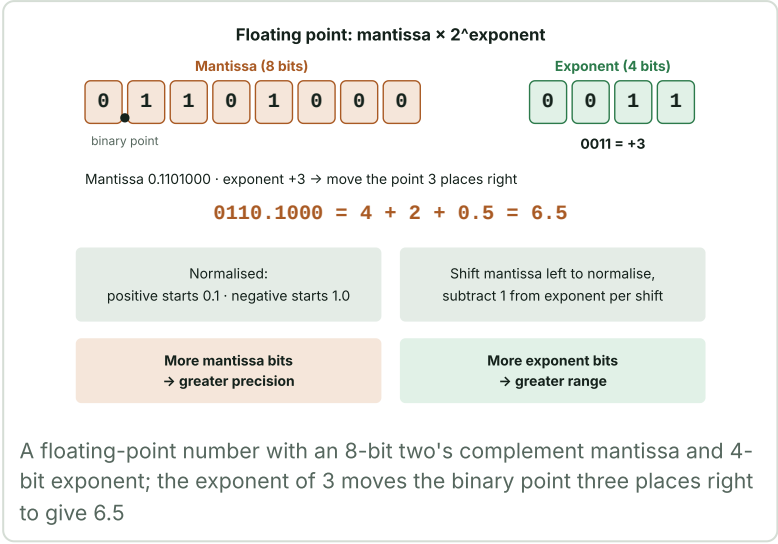
Real numbers: fixed-point and floating-point

FD03
FACT FILE

Specification: students should be able to

- Represent real numbers in fixed-point binary and convert between fixed-point and denary
- Explain floating-point representation using a two's complement mantissa and exponent
- Convert between denary and normalised floating-point binary
- Normalise floating-point numbers and explain the purpose of normalisation
- Explain the trade-off between range and precision and compare fixed-point with floating-point
- Describe rounding errors, overflow and underflow with real numbers

Key words: real number, fixed point, binary point, floating point, mantissa, exponent, normalisation, precision, range, rounding error, absolute error, relative error, overflow, underflow, IEEE 754



A floating-point number with an 8-bit two's complement mantissa and 4-bit exponent; the exponent of 3 moves the binary point three places right to give 6.5

Why real numbers need special treatment

Many quantities that programs handle are not whole numbers: distances, temperatures, prices, times and scientific measurements. A computer stores everything in a fixed number of bits, so it needs an agreed way to represent the fractional part. Binary place value can be extended to the right of a binary point, where the columns are worth a half, a quarter, an eighth, a sixteenth and so on. There are two ways to decide where that point goes. In fixed-point representation the position of the point is agreed in advance and never changes. In floating-point representation the number is stored as a mantissa and an exponent, and the exponent says how far to move the point, so it can 'float' to suit the size of the number. Both methods trade off range, which is how large and how small the stored values can be, against precision, which is how finely values can be distinguished. Neither can represent every real number exactly, because there are infinitely many real numbers and only a finite number of bit patterns. Understanding these limits explains why programs sometimes give answers such as 0.9999999999999999 instead of 1.

Fixed-point representation

In fixed-point binary a set number of bits represent the whole part and the rest represent the fraction. With four bits either side of the point, 0110.1010 means 4 + 2 + 0.5 + 0.125 = 6.625. To convert denary to fixed point, split the value into powers of two: 9.375 is 8 + 1 + 0.25 + 0.125, which is 1001.0110. Negative values can be stored by using two's complement, so that in this format the leftmost bit is worth −8. The precision is the value of the smallest column, here 1/16 or 0.0625, and it is the same across the whole range, which is described as constant absolute precision. The range is limited: the largest unsigned value with four whole-number bits is 15.9375. Moving the point to the right increases the range but leaves fewer bits for the fraction, so precision falls; moving it left does the reverse. Fixed point is simple and fast because the processor can use ordinary integer circuits, which makes it suitable for systems with a known, narrow range, such as audio samples, some sensors and embedded controllers.

Floating-point representation

Floating point is the binary version of standard form. A number is stored as a mantissa multiplied by two raised to the power of an exponent. In the format used in this course both parts are in two's complement. The mantissa is a fraction with the binary point immediately after its first bit, which is the sign bit and is worth −1; the exponent is a two's complement integer. To convert to denary, first work out the exponent, then move the binary point that many places – right for a positive exponent, left for a negative one – and finally add up the place values, remembering that the leftmost bit of the result has a negative weight if the mantissa is negative. For example, 0.1101000 with exponent 0011 becomes 0110.1000, which is 6.5. Floating point allows a very wide range of values, from tiny fractions to huge numbers, to be stored in the same number of bits. The cost is that the arithmetic is more complex and slower than for fixed point, and the precision is relative: large numbers are stored with large gaps between neighbouring values.

Normalisation

The same value can be written in floating point in more than one way. For example, 0.0010110×2^{6} and 0.1011000×2^{4} both equal 11. The first version wastes two bits of the mantissa on leading zeros that add no information. A floating-point number is normalised when the first two bits of the mantissa differ: 0.1 for positive numbers and 1.0 for negative numbers. To normalise, shift the mantissa left until this is true, filling with zeros on the right, and subtract one from the exponent for every place shifted, so that the value is unchanged. For a negative number, the redundant bits are repeated leading 1s: 1.1101000×2^{3} becomes 1.0100000×2^{1} after two shifts. Normalisation has two benefits. It makes the most of the available mantissa bits, so the precision is as high as possible, and it ensures each value has exactly one representation, which makes comparing and processing numbers simpler. When converting denary to floating point, aim for a normalised result directly by moving the point until the number starts 0.1, then counting the places moved to get the exponent.

Range and precision

With a fixed total number of bits, every bit given to the mantissa is a bit taken from the exponent. More mantissa bits increase precision: more significant bits are kept, so values can be stored more exactly and the gaps between representable numbers are smaller. More exponent bits increase range: the binary point can be moved further, so both very large and very small magnitudes can be stored. With an 8-bit mantissa and a 4-bit exponent the largest positive value is 0.1111111×2^{7}, which is 127, and the smallest positive normalised value is 0.1×2^{-8}, about 0.00195. Re-dividing the same twelve bits as six and six raises the largest value to just under 2^{31}, but leaves only five significant bits after the sign. Designers choose the split to suit the application: scientific calculations need a wide range, while measurements that must be distinguished finely need precision. Real computers use the IEEE 754 standard, where a 32-bit float uses 8 exponent bits and 23 mantissa bits and a 64-bit double uses 11 and 52. It differs in detail from the exam format but follows the same trade-off.

Rounding errors, overflow and underflow

Many simple denary fractions cannot be stored exactly in binary. Just as one third recurs in denary, one tenth recurs in binary as 0.000110011..., so it must be cut off and stored as the nearest available value. The difference between the true and stored value is the absolute error; dividing it by the true value gives the relative error. In a 4.4 fixed-point format, 0.1 would be rounded to the nearest step, 0.125, an absolute error of 0.025 and a relative error of 25 per cent. Errors grow when calculations are repeated: adding 0.1 ten times in a C# double gives 0.9999999999999999, so a test for equality with 1.0 fails. For this reason programmers compare real numbers within a small tolerance, and financial software stores money as whole pence or uses a decimal type. Overflow happens when a result's magnitude is too large for the exponent field to hold. Underflow happens when a result is so close to zero that the exponent cannot be made negative enough, so it is stored as zero. Both can cause wrong results or errors if they are not detected.

Key terms from the mark schemes

Real number
A number that can have a fractional part, such as 6.75 or −0.3125.

Binary point
The point separating the whole-number bits from the fractional bits, whose place values are ½, ¼, ⅛ and so on.

Fixed-point representation
A way of storing real numbers in which the binary point is at an agreed, unchanging position.

Floating-point representation
A way of storing real numbers as a mantissa multiplied by 2 to the power of an exponent, allowing a wide range of values.

Mantissa
The part of a floating-point number that holds its significant digits.
Credited: significand

Normalisation
Adjusting a floating-point number so the first two bits of the mantissa differ, maximising precision and giving a unique representation.

Range (floating point)
The span between the largest and smallest magnitudes that can be represented, determined mainly by the exponent bits.

Underflow
An error that occurs when a value is too close to zero to be represented with the available exponent.

Exponent
The part of a floating-point number that states how many places, and in which direction, the binary point is moved.

Precision
How finely values can be distinguished, determined by the number of significant bits (mantissa bits).

Rounding error
The difference between a real value and the nearest value that can be stored in the available bits.

C C# example

Decoding floating point in C#

```
string m = "01101000";   // mantissa 0.1101000
string e = "0011";       // exponent
double mant = -(m[0] - '0');          // sign bit is worth -1
for (int i = 1; i < m.Length; i++)
    mant += (m[i] - '0') * Math.Pow(2, -i);
int exp = Convert.ToInt32(e, 2);
if (e[0] == '1') exp -= 1 << e.Length;   // two's complement exponent
Console.WriteLine(mant * Math.Pow(2, exp)); // 6.5
```

Exam focus

Floating-point conversions are marked step by step, so write the exponent in denary first, then show the mantissa with the binary point moved, then the place-value sum and the final answer. Remember that the leftmost bit of a negative mantissa is negative, and that a negative exponent moves the point left. When asked to normalise, give both the new mantissa and the new exponent in binary, padded to the stated number of bits, and check that the value has not changed. When converting a negative denary value, find the positive mantissa first, then take its two's complement; the exponent does not change. Questions about changing the split of bits need the direction of change for both range and precision, for example 'more exponent bits increase the range but reduce precision because there are fewer mantissa bits'. 'Explain why numbers are normalised' needs both maximum precision and a unique representation. Avoid vague claims that one method is 'more accurate' without linking it to the number of significant bits, and do not confuse underflow with a negative number.

? Check yourself

1. State what is meant by the mantissa of a floating-point number. [1]
2. Convert the fixed-point number 0101.1100 (4 bits before and 4 bits after the binary point) to denary. Show your working. [2]
3. Represent 3.625 in unsigned fixed-point binary using 4 bits before and 4 bits after the binary point. [2]

Answers: 1. The part of a floating-point number that holds the significant digits/bits of the value (1) 2. 4 + 1 + 0.5 + 0.25 shown (1) / = 5.75 (1) 3. 3 = 0011 (1) / .625 = .1010, giving 0011.1010 (1)

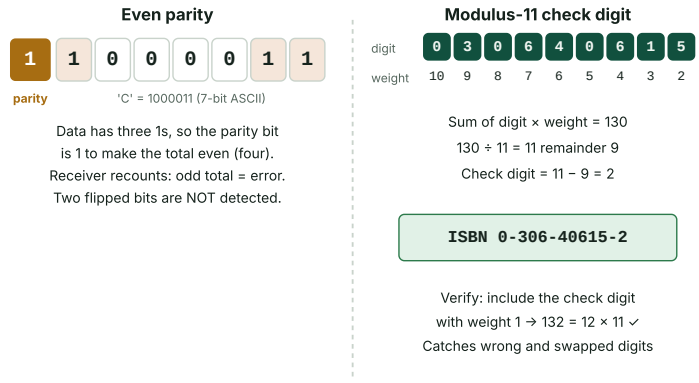
Character sets and error detection

FD04
FACT FILE

Specification: students should be able to

- Explain what a character set is and why an agreed standard is needed for text to be exchanged between computers
- Demonstrate how ASCII, extended ASCII and Unicode are used to represent characters
- Compare character sets in terms of the number of characters represented and the storage required
- Describe how odd and even parity bits are used to detect transmission errors, and their limitations
- Explain the purpose of a check digit and how it differs from a parity bit

Key words: character set, character code, control character, ASCII, extended ASCII, code page, Unicode, code point, UTF-8, parity bit, even parity, odd parity, check digit, Modulus 11, error detection, transposition error, checksum, echo check



An even parity bit added to the ASCII code for C, and a modulus-11 check digit calculation for an ISBN-10

Text as numbers: an overview

Everything a computer stores is binary, so text has to be turned into numbers before it can be saved, processed or sent. A character set is the agreed list of characters a system can represent, with a unique binary code for each one. Because the same code must mean the same character on every machine, standards are essential: a message typed on a phone in Omagh must appear identically on a laptop in Lisbon. The oldest widely used standard is ASCII, a 7-bit code with 128 characters. Extended ASCII uses the eighth bit to give 256 characters, and Unicode goes much further, giving a code point to characters from almost every writing system plus symbols and emoji. Because data can also be corrupted as it travels or as it is typed in, systems add extra information to help spot mistakes. A parity bit added to each byte can reveal that a bit has flipped during transmission, and a check digit added to a code, such as one calculated with the Modulus 11 method, can reveal that a number has been keyed in wrongly. This lesson covers both ideas: how characters are represented, and how simple redundancy is used to detect errors.

How ASCII is organised

ASCII, the American Standard Code for Information Interchange, uses 7 bits, so it has 2 to the power 7, or 128, codes numbered 0 to 127. Codes 0 to 31 are control characters that do not print, such as tab, line feed and carriage return, and code 127 is delete. Code 32 is the space, codes 48 to 57 are the digits, 65 to 90 are the capital letters and 97 to 122 are the lower-case letters. Because the codes are allocated in order, you can work out an unknown code from a known one: if 'A' is 65 then 'H' is 72, and each lower-case letter is exactly 32 more than its capital, which in binary means only one bit is different. The ordering also lets programs sort text by comparing codes, although it means capitals sort before all lower-case letters. It is important to distinguish a digit character from a number: the character '5' is stored as code 53, so arithmetic cannot be done on it until it is converted. A plain text file stores one byte per ASCII character, so a five-letter word saved on its own occupies five bytes, while a word-processed file is larger because formatting is saved too.

Extended ASCII and Unicode

ASCII has no accented letters, so it cannot store an Irish fada, a French cedilla or a German umlaut, and it has nothing for non-Latin scripts. Extended ASCII uses all eight bits of the byte, giving 256 codes. The first 128 are identical to ASCII with a leading zero added, and codes 128 to 255 hold extra characters. The problem is that there is no single extended ASCII: different code pages give those upper codes different meanings, so a file created with one code page can show the wrong symbols when opened with another. Unicode solves this by giving every character one agreed code point that is the same worldwide. It has room for more than 1.1 million code points, and its first 128 match ASCII so existing ASCII text remains valid. Unicode is stored using an encoding. UTF-8 is the most common on the web: it uses one byte for ASCII characters and two, three or four bytes for others. UTF-16 uses two or four bytes and UTF-32 always uses four. Unicode is now built into all major operating systems, programming languages, browsers and email systems, and new characters such as emoji are added regularly.

Comparing storage requirements

The choice of character set affects both what can be represented and how much storage and bandwidth text needs. ASCII and extended ASCII always use one byte per character, so a 2,000-character document takes 2,000 bytes. In UTF-8, an English document of the same length is also 2,000 bytes because every ASCII character still needs just one byte, which is why UTF-8 is so popular. A document full of accented letters grows, since each of those needs two bytes; for example, the 12-character name 'Seán Ó Néill' needs 15 bytes in UTF-8 but 12 in a Western European code page. Chinese characters typically need three bytes each in UTF-8. UTF-16 doubles the size of English text and UTF-32 quadruples it, but fixed-width encodings make it quicker to jump to the nth character. A file containing thousands of different Chinese characters must therefore use Unicode, because 256 codes cannot represent them, and the trade-off is a larger file. When answering, link the number of bits to the number of characters using 2 to the power n, and link the extra bytes to larger files and longer transmission times.

Parity checking

When data is sent along a cable or over a wireless link, electrical interference can flip a bit from 0 to 1 or 1 to 0. A parity bit is a simple way of detecting this. One extra bit is added to each group of data bits, commonly turning a 7-bit ASCII code into a byte. With even parity, the parity bit is chosen so that the byte, parity bit included, holds an even count of 1s; with odd parity that count is odd. Both ends must agree beforehand which system is used. The receiver counts the 1s in every byte it receives; if the count does not match the agreed parity, it knows the byte has been corrupted and asks for it to be sent again. Parity has clear limitations. It cannot tell which bit is wrong, so it cannot correct the error itself. If two bits, or any even number of bits, are flipped in the same byte, the count of 1s stays even or odd and the error passes unnoticed. It also adds overhead, since one bit in every eight carries no data. More robust methods such as checksums and cyclic redundancy checks are used for larger blocks of data.

Check digits compared with parity

Parity and check digits both add redundant information so that errors can be spotted, but they protect data at different stages. A parity bit is attached to every byte that is transmitted, and it detects bits flipped by interference on the way. A check digit is attached to a code number, such as a book number, a barcode or a membership number, and it detects mistakes made when that number is typed in or scanned. The check digit is calculated from the other digits; whenever the code is entered, the system repeats the calculation and rejects the code if its answer does not match the digit supplied. The Modulus 11 method, used for old ten-digit ISBNs, gives each digit a different weighting, so it catches every single wrong digit and every swap of two digits, known as a transposition error. Its full method, including the special case where X stands for 10, is covered in FD07. Neither technique can correct an error, and a valid check digit only shows that a code is well formed, not that it belongs to the right person or item.

T Key terms from the mark schemes

Character set
An agreed list of characters that a computer system can represent, in which each character is given a unique binary code.
`Credited: A defined list of characters recognised by the hardware and software`

ASCII
American Standard Code for Information Interchange: a 7-bit character set of 128 codes covering English letters, digits, punctuation and control characters.

Extended ASCII
An 8-bit extension of ASCII giving 256 codes; codes 0–127 match ASCII and codes 128–255 hold accented letters and symbols, which vary between code pages.

UTF-8
A variable-length Unicode encoding that uses one byte for ASCII characters and up to four bytes for other characters.

Parity bit
An extra bit added to a group of bits so that the total number of 1s is even (even parity) or odd (odd parity), allowing transmission errors to be detected.

Modulus 11
A check digit method in which each digit is multiplied by a weighting, the products are added, and the check digit is found from the remainder after dividing by 11 (0 gives 0, 1 gives X, otherwise 11 minus the remainder).
Credited: Mod 11

Unicode
A character standard that gives a unique code point to characters from almost all of the world's writing systems, plus symbols and emoji; its first 128 codes match ASCII.

Control character
A non-printing character code, such as carriage return, line feed or tab, used to control how text is processed or displayed.

Check digit
An extra digit calculated from the other digits of a code number and added to the end, so the computer can detect entry or reading errors by recalculating it.

Transposition error
An input error in which two characters are entered in the wrong order, such as 47019 instead of 40719.

C C# example

Characters as numbers in C#

```
char letter = 'A';
int code = letter;                      // implicit conversion: 65
Console.WriteLine(code);

char lower = (char)(code + 32);         // 97 is 'a'
Console.WriteLine(lower);

string bits = Convert.ToString(code, 2).PadLeft(8, '0');
Console.WriteLine(bits);                // 01000001

Console.WriteLine("7" + "1");           // text joins: 71
Console.WriteLine('7' - '0');           // 55 - 48 = 7
```

Exam focus

Expect short State and Describe questions on character sets and parity, plus short calculations. When working with ASCII codes, use the ordering of the table to find unknown codes and show the addition or subtraction you used, for example 'K' is 75 so 'N' is 78. For parity, always say whether you are using odd or even, count the 1s explicitly, state the parity bit and give the complete byte. Limitations must be specific: say that two flipped bits leave the parity unchanged, not just that parity is unreliable, and that the receiver cannot tell which bit is wrong. When comparing ASCII and Unicode, quote the number of characters (128, 256, over a million) and give a consequence, such as more languages but more bytes per character. A common error is to claim that UTF-8 always uses two bytes; it uses one to four. If you are asked to compare parity with a check digit, link parity to transmission errors and check digits to data-entry errors. Full Modulus 11 calculations are examined with validation, so practise them using FD07, and do not confuse a check digit, which validates input, with verification by double entry.

? Check yourself

1. State what is meant by a character set. [2]
2. The ASCII code for 'K' is 75. State the ASCII code for 'N' and the ASCII code for 'k'. [2]
3. A pupil stores the digit '5' in a string variable and the number 5 in an integer variable. Explain why these are stored differently. [2]

Answers: 1. An agreed set of characters that a computer system can recognise and represent (1) / each character has its own unique binary code/number (1). 2. 'N' = 78 (1) / 'k' = 107 (1). 3. The character '5' is stored as its character code (53 in ASCII/Unicode) (1) / the integer 5 is stored as the binary value of 5 (00000101), so arithmetic can only be done directly on the integer (1).

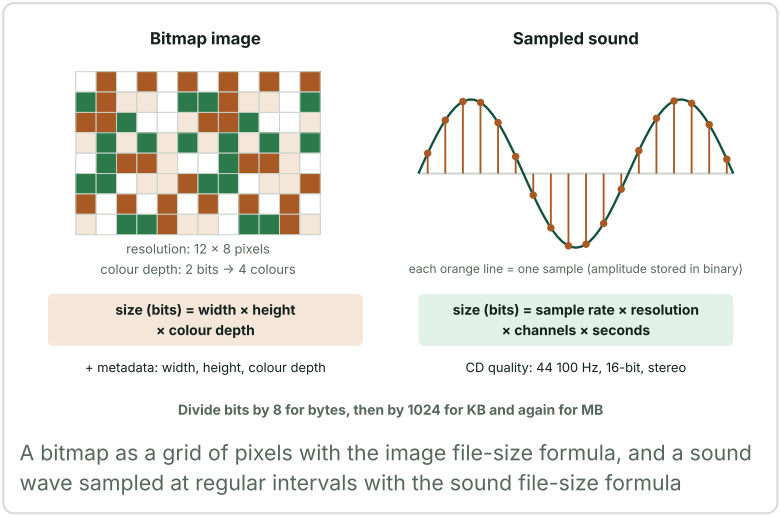
Representing images and sound

FD05
FACT FILE

Specification: students should be able to

- Describe how bitmap images are represented using pixels, resolution, colour depth and the RGB colour model
- Explain the purpose of metadata stored with an image file
- Compare bitmap and vector graphics and select the appropriate type for a task
- Describe how sound is digitised by sampling, including sampling rate and sample resolution
- Calculate the file sizes of uncompressed images and sound files and convert between units

Key words: pixel, bitmap, resolution, dots per inch (dpi), colour depth, RGB, metadata, vector graphic, pixelation, analogue, sampling, sampling rate, sample resolution (bit depth), analogue-to-digital converter (ADC), channel, Nyquist theorem, file size



A bitmap as a grid of pixels with the image file-size formula, and a sound wave sampled at regular intervals with the sound file-size formula

Digitising images and sound: an overview

Light and sound in the real world are analogue: they vary smoothly and continuously. A computer can only store discrete binary values, so images and sound must be digitised by measuring them and storing each measurement as a number. A bitmap image is digitised in space: the picture is divided into a grid of pixels and the colour of each pixel is recorded using a fixed number of bits, called the colour depth. Sound is digitised in time: an analogue-to-digital converter measures the amplitude of the sound wave thousands of times every second, and each sample is stored using a fixed number of bits, called the sample resolution. In both cases the same trade-off appears. Taking more measurements, through a higher resolution or a higher sampling rate, or storing each measurement more precisely, through a greater colour depth or sample resolution, makes the digital version a closer match to the original, but it also increases the file size, the storage needed and the time taken to transmit the file. Because the uncompressed size can be calculated exactly from these settings, file size calculations are a regular feature of exam papers. Vector graphics take a different approach, describing shapes mathematically rather than recording individual pixels.

Pixels, resolution and colour depth

A bitmap image is a two-dimensional grid of pixels, each of which stores one colour as a binary value. The resolution is the number of pixels in the image, usually written as width by height, such as 1920 by 1080. For printed work, resolution is often given in dots per inch: screen graphics are typically around 72 dpi, desktop printing around 150 dpi and professional printing 300 dpi or more. To find the pixel dimensions of a printed image, multiply each measurement in inches by the dpi. Colour depth is the number of bits used for each pixel, and n bits allow 2 to the power n colours, so 8 bits give 256 colours and 24 bits give 16,777,216. Most screens build colours from red, green and blue light, and 24-bit colour uses one byte for each of these three components, with values from 0 to 255. Increasing the resolution adds more pixels and therefore more detail, while increasing the colour depth allows smoother, more realistic shading. Both increase the file size, because the total number of bits is the number of pixels multiplied by the bits per pixel. If a bitmap is enlarged beyond its resolution, the individual pixels become visible, an effect called pixelation.

Metadata

The pixel data on its own is just a long sequence of bits, so an image file also stores metadata, which is data about the data. The most important items are the width and height of the image in pixels and its colour depth; without these, software could not tell where one row of pixels ends and the next begins, or how many bits belong to each pixel, and the picture could not be reconstructed correctly. Other metadata can include the resolution in dpi, the file format, the type of compression used and, for some formats such as GIF, the colour palette. Digital cameras and smartphones add further information known as EXIF data, such as the date and time a photo was taken, the camera model, exposure settings and frequently the GPS coordinates of the location. This extra information is useful for organising and searching photo collections, but it has privacy implications: a photograph shared online could reveal where a person lives or where a child goes to school, which is why many social media platforms remove location data when an image is uploaded. Metadata increases the file size slightly, but exam calculations normally ignore it unless the question says otherwise.

Bitmap and vector graphics compared

A vector graphic is not stored as pixels at all. Instead, it is a list of drawing objects, such as lines, circles, rectangles, curves and text, each described by properties including coordinates, size, line width, line colour and fill colour. Whenever the image is displayed or printed, the software recalculates the shapes at the required size, so a vector can be scaled up indefinitely without losing sharpness. A bitmap, by contrast, has a fixed number of pixels and becomes blocky when enlarged. The size of a vector file depends on how many objects it contains, not on how large it is displayed, so a company logo stored as a vector is usually very small and can be used on a business card or the side of a lorry from the same file. Vectors are also easy to edit, because each object can be moved, resized or recoloured independently. However, they are not suited to photographs, where colour changes continuously from pixel to pixel and cannot be described efficiently as shapes. Bitmaps are therefore used for photographs, scanned documents and realistic artwork, while vectors are preferred for logos, signs, icons, fonts, maps, charts and diagrams. SVG is the common vector format on the web.

Sampling sound

Sound travels as a pressure wave, and a microphone converts it into an analogue electrical signal whose voltage varies continuously. An analogue-to-digital converter, or ADC, measures this signal at regular, equally spaced intervals; each measurement is a sample of the wave's amplitude and is stored as a binary number. The sampling rate is the number of samples taken each second, measured in hertz, and CD-quality audio uses 44,100 samples per second. A higher sampling rate captures the shape of the wave more closely and allows higher frequencies to be recorded. The Nyquist theorem states that the sampling rate must be at least twice the highest frequency that needs to be reproduced; because human hearing extends to about 20 kHz, a rate a little over 40 kHz is sufficient. The sample resolution, or bit depth, is the number of bits per sample. With 16 bits there are 65,536 possible amplitude levels, so each sample is rounded to the nearest level; more bits mean less rounding error and a more faithful recording. Stereo recordings store two channels, one for each speaker. On playback, a digital-to-analogue converter turns the stored samples back into a varying voltage.

Calculating file sizes

Uncompressed file sizes follow directly from the way the data is represented. For a bitmap image, the size in bits is the width in pixels multiplied by the height in pixels multiplied by the colour depth. If dimensions are given in inches, first multiply each by the dpi to find the pixel width and height. For sound, the size in bits is the sampling rate multiplied by the sample resolution, the number of channels and the duration in seconds; remember to convert minutes into seconds first. Once you have the number of bits, divide by 8 to get bytes, by 1024 to get kilobytes, and by 1024 again for megabytes. Some questions, and some textbooks, use 1000 instead of 1024; the convention used by the question should be followed, and if none is stated, showing which you have used gains credit. Calculations can also be reversed: for example, the number of photos that fit on a memory card is the card's capacity in bytes divided by the size of one photo, rounded down because a partial photo cannot be stored. These are the sizes of uncompressed formats such as BMP and WAV; compressed formats such as JPEG and MP3 are far smaller because they discard or encode data more efficiently.

T Key terms from the mark schemes

Pixel
Short for picture element: the smallest addressable element of a bitmap image, storing a single colour as a binary value.

Colour depth
The number of bits used to represent the colour of each pixel; n bits give 2^n possible colours, and more bits give a larger file.
Credited: Bit depth

Bitmap
An image stored as a grid of pixels, with the colour of each pixel held as a binary number.
Credited: Raster image

Sampling rate
The number of samples of an analogue sound wave taken per second, measured in hertz.
Credited: Sample rate / sampling frequency

Analogue-to-digital converter (ADC)
A device that measures an analogue signal at regular intervals and converts each measurement into a binary value.

Pixelation
The blocky appearance of a bitmap image when it is enlarged beyond its resolution so that individual pixels become visible.

Resolution
The number of pixels in an image, usually given as width × height in pixels, or as dots per inch (dpi) for printed images.
Credited: Pixel density / dpi

Metadata
Data about data, such as an image's width, height and colour depth, stored with the file so that software can interpret it correctly.

Vector graphic
An image stored as a set of objects described by mathematical properties such as coordinates, size and colour, so it can be scaled without loss of quality.

Sample resolution
The number of bits used to store each sound sample, which determines how precisely each amplitude is recorded; also called bit depth.
Credited: Bit depth

Nyquist theorem
The principle that the sampling rate must be at least twice the highest frequency in the signal for it to be reproduced accurately.

C# example

Checking file sizes in C#

```
long width = 1024, height = 768, depth = 24;
long imageBits = width * height * depth;
double imageMB = imageBits / 8.0 / 1024 / 1024;
Console.WriteLine("Image: " + imageMB + " MB");          // 2.25

long rate = 44100, res = 16, channels = 2, secs = 180;
long soundBits = rate * res * channels * secs;
double soundMB = soundBits / 8.0 / 1024 / 1024;
Console.WriteLine("Sound: " + Math.Round(soundMB, 1) + " MB");   // 30.3
```

Exam focus

Calculation questions typically carry two to four marks, with method marks for each stage, so always write the formula, substitute the values and show every unit conversion; a final answer on its own risks losing most of the marks if it is wrong. Check what unit the question asks for and whether it expects 1000 or 1024. Common mistakes are forgetting to divide bits by 8, forgetting to convert minutes to seconds, leaving out the number of channels for stereo, and confusing colour depth in bits with the number of colours. When asked how many colours a bit depth allows, show 2 to the power n. For 'explain the effect' questions, give a chain of reasoning: a greater colour depth means more bits per pixel, multiplied across every pixel, so a larger file but more colours and smoother shading. For bitmap versus vector, use paired comparison points covering how the image is stored, what happens when it is scaled, what file size depends on and what each is suited to, and apply them to the context, such as a logo printed at many sizes. Metadata answers should name specific items like width, height and colour depth and say why software needs them.

? Check yourself

1. State what is meant by the colour depth of an image. [1]
2. State how many different colours can be represented with a colour depth of 16 bits. [1]
3. Calculate the file size, in kilobytes, of an uncompressed 640 × 480 pixel image with a colour depth of 8 bits. Show your working. [3]

Answers: 1. The number of bits used to represent the colour of each pixel (1). 2. 2^{16} (1) = 65,536 colours. [1 mark for 65,536] 3. 640 × 480 = 307,200 pixels (1) / × 8 bits = 2,457,600 bits ÷ 8 = 307,200 bytes (1) / ÷ 1024 = 300 KB (1).

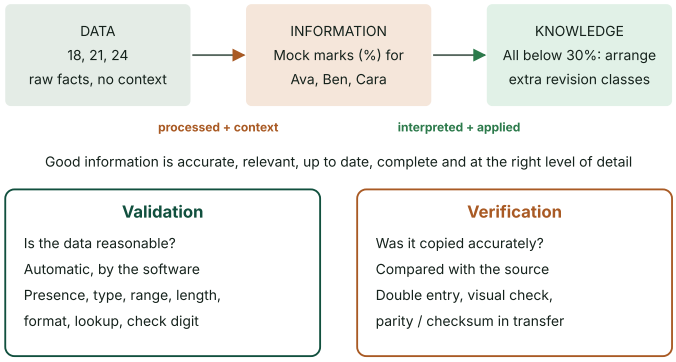
Data, information and knowledge; quality of information; validation and verification

FD06 FACT FILE

Specification: students should be able to

- Distinguish between data, information and knowledge by using examples
- Describe how the quality of information is affected by accuracy, relevance, being up to date, completeness, presentation and reliability
- Describe and apply the range, type, length, format, presence and lookup validation checks
- Explain the purpose of the verification methods double entry and proofreading
- Explain the limitations of data validation and data verification

Key words: data, information, knowledge, context, GIGO, transcription error, transposition error, accuracy, relevance, currency (up to date), completeness, reliability, validation, range check, format check, lookup check, verification, double entry



Data becomes information when it is processed and given context, and information becomes knowledge when it is interpreted and applied; validation and verification compared

Data, information and knowledge

Data is a collection of raw facts and figures – numbers, text, codes, dates, images or sounds – that has no context and therefore no meaning on its own. The sequence 11.2, 9.6, 12.4 could be anything. When data is processed and given context, for example by labelling it as the bleep-test levels of three under-16 rugby players, it becomes information, which has meaning for the person using it. This is often summarised as information = data + context + meaning. Knowledge goes one step further: it is produced by applying a rule, or a set of rules, to information so that an informed decision can be made. If the coach's rule is that any player below level 10 needs extra conditioning, then knowing that one particular player must join that group is knowledge. The distinction matters because organisations collect data in order to produce information, and they use information, together with their own rules and experience, to make decisions. In an answer, always show all three levels using the same example so that the progression from raw values, to meaningful information, to a decision based on a rule is clear.

Quality of information and GIGO

The quality of information depends on the quality of the data used to produce it, an idea summed up as 'garbage in, garbage out' (GIGO). Errors creep in in several ways. A transcription error happens when data is mistyped, such as 1500 entered instead of 150. A transposition error happens when two characters swap places, such as 40719 becoming 47019. Spreadsheets may contain incorrect formulae, which can be caught by calculating important totals in two different ways and comparing them. Surveys may produce biased or meaningless results if questions are poorly worded or only one group of people is asked. Good-quality information has six characteristics: it is accurate, relevant to the user and purpose, up to date, complete enough for the decision, well presented in a suitable format, and reliable because it comes from a trustworthy, verified source. Using poor information can be worse than having none, because people act on it with false confidence and may make costly or dangerous decisions.

Value and cost of information

Information is valuable to an organisation when it leads to better decisions, lower costs, better customer service or an advantage over competitors. A supermarket that knows which products sell on which days can order the right stock and reduce waste; a hospital that knows its bed occupancy can plan staff rotas. The value of information falls if it arrives too late to influence a decision, if it is inaccurate or incomplete, or if competitors have the same information.

Information also has a cost: collecting the data, entering and checking it, storing it securely, keeping it up to date and training staff to use it all cost money. An organisation should therefore only collect information whose value is greater than its cost. Where information is about identifiable people, it is personal data, and it must be processed lawfully, fairly and securely under the UK GDPR and the Data Protection Act 2018, which also requires that data is accurate, kept up to date and not held for longer than necessary.

Validation checks

Validation is an automatic check made by software when data is entered, to confirm that the input obeys set rules and is reasonable and sensible. A range check makes sure a value lies between a lower and an upper limit, such as a shoe size from 1 to 15. A type check makes sure the data is of the correct type, such as an integer for a number of tickets. A length check makes sure the right number of characters has been entered, such as exactly 11 characters for a mobile number. A format check makes sure the data matches a pattern, such as two letters followed by four digits. A presence check makes sure a field has not been left blank, which is why online forms mark compulsory fields with an asterisk. A lookup check compares the entry with a list or file of valid values, such as checking that a membership number exists in the members file. Often several checks are needed on one field, because a format check alone would accept an impossible date such as 45/33/87.

Verification methods

Verification checks that data has been entered or copied accurately, so that every copy exactly matches the original source. In double entry, the data is entered twice and the computer compares the two versions, flagging any difference; this is why websites ask you to type a new password or email address twice. Double entry catches most typing mistakes but doubles the workload, so for large volumes it is usually kept for the most important fields. In proofreading, or visual checking, the data on screen or on a printout is compared by eye with the source document. It works best when a different person does the checking, as in a pharmacy where a second pharmacist checks the printed label against the prescription. Proofreading is slow and expensive for large amounts of data and errors can be overlooked when people rush. Many systems also display the entered details and ask the user to confirm them, and backup software verifies that the copy matches the original.

Limitations of validation and verification

Validation and verification reduce errors but cannot remove them completely. Validation only checks that data is reasonable, so a value that is sensible but wrong will be accepted: an age of 34 typed instead of 43 passes any sensible range check. Some impossible values can also slip through if the rules are too simple; a day-of-month check of 1 to 31 accepts 31 April.

Verification only checks that the copy matches the source. If the original paper form contains a mistake, or the customer gave the wrong information, then the stored data will be an exact copy of a wrong value. Double entry can be defeated when the person makes the same mistake twice, and proofreading depends on the concentration of the checker. Because of these limits, organisations combine methods: validation rules at the point of entry, verification of the most important fields, and processes for people to check and correct their own records, which is also a right under data protection law.

T Key terms from the mark schemes

Data
Raw facts and figures, such as numbers, text, images or sound, that have no context and therefore no meaning on their own.
`Credited: Raw facts and figures without context`

Information
Data that has been processed and given context so that it has meaning to the person using it.
`Credited: Data + context + meaning`

Knowledge
The result of applying one or more rules to information so that an informed decision can be made.
`Credited: Information that has been interpreted and applied using experience or rules`

GIGO
Garbage in, garbage out: if the input data is of poor quality, the output information will also be of poor quality.

Transposition error
An input error in which two characters are entered in the wrong order, such as 47019 instead of 40719.

Transcription error
An input error in which data is copied or typed incorrectly, such as 1500 instead of 150.

Validation
An automatic check carried out by software to make sure that input data is sensible and reasonable and obeys set rules.
Credited: Checks data is within acceptable boundaries

Range check
A validation check that a value lies between a lower and an upper limit.

Format check
A validation check that data matches a set pattern of characters, such as two letters followed by four digits.
Credited: Picture check

Lookup check
A validation check that compares the entered value with a list or file of valid values.
Credited: List check

Verification
A check that data has been entered or copied accurately, so that it exactly matches the original source.

Double entry
A verification method in which data is entered twice and the computer compares the two versions, flagging any difference.

C C# example

Validation in C#

```
string[] forms = { "11A", "11B", "11C" };
Console.Write("Form class: ");
string form = Console.ReadLine() ?? "";
if (form == "") Console.WriteLine("Presence check failed");
else if (Array.IndexOf(forms, form) == -1) Console.WriteLine("Lookup check failed");
else Console.WriteLine("Form class accepted");

Console.Write("Year group (8-14): ");
string input = Console.ReadLine() ?? "";
if (input == "") Console.WriteLine("Presence check failed");
else if (!int.TryParse(input, out int year)) Console.WriteLine("Type check failed");
else if (year < 8 || year > 14) Console.WriteLine("Range check failed");
else Console.WriteLine("Year group accepted");
```

Exam focus

Questions on this lesson usually use State, Describe, Explain or Compare and carry one mark per valid point. For data, information and knowledge, mark schemes credit a definition plus an example for each, ideally taken from the table or scenario given: raw value, value with context, then a rule applied to reach a decision. Knowledge answers that do not mention applying a rule usually lose the mark. For quality factors, name the factor and explain its effect in the context, for example that an incomplete re-order message does not say which pack size to buy. For validation, name the check and give the precise rule with the field and limits, such as 'range check: hour >= 0 and <= 23'. 'Checks the data is correct' is never credited for validation; say 'sensible' or 'reasonable'. For verification, describe how the method works and a drawback. Common mistakes are confusing a type check with a format check, describing double entry as validation, and claiming that either method guarantees accuracy.

? Check yourself

1. A hospital ward records patients' temperatures. Using an example from this ward, show how data, information and knowledge differ. [3]
2. Explain what is meant by 'garbage in, garbage out' (GIGO). [2]
3. A pharmacy technician enters a patient number of 40719 as 47019. State the type of error that has been made and explain why validation may not detect it. [3]

Answers: 1. Data: a raw value with no context, e.g. 38.9 (1) / Information: the data with context/meaning, e.g. patient in bed 4 had a temperature of 38.9 °C at 14:00 (1) / Knowledge: a rule applied to the information to make a decision, e.g. rule 'above 38 °C means fever', so the nurse alerts the doctor (1) 2. If the input data is of poor quality / inaccurate (1) / the output information will also be poor / inaccurate, however good the processing system is (1) 3. Transposition error (1) / 47019 is still a sensible / reasonable patient number, e.g. correct length, type and format (1) / validation only checks data is reasonable, not that it is correct (1). Credit: a check digit could detect it.

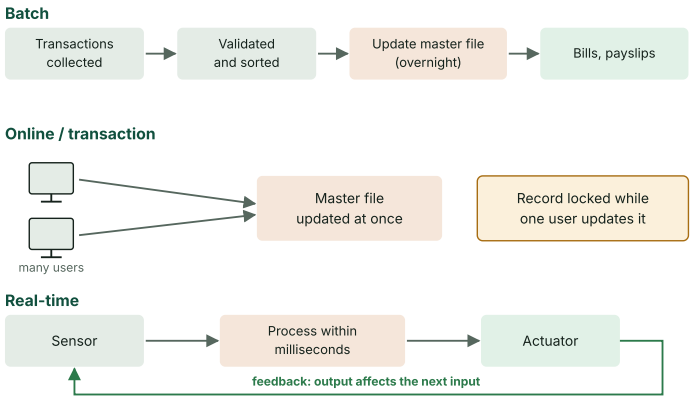
Check digits, batch totals, data capture and processing methods

FD07 FACT FILE

Specification: students should be able to

- Explain the purpose of check digits and calculate and check a Modulus 11 check digit
- Explain the purpose of batch totals, control totals and hash totals and apply them to a batch of documents
- Describe manual and automated methods of data capture
- Describe batch and real-time processing, distinguishing real-time transaction (online) processing from real-time control, and select an appropriate method for a given application
- Explain multitasking, multi-user and multithreading in the context of processing

Key words: check digit, Modulus 11, weighting, remainder, batch total, control total, hash total, data capture, optical mark recognition (OMR), batch processing, master file, transaction file, real-time transaction (online) processing, real-time control, record locking, multitasking, multi-user, multithreading



Batch processing collects transactions and updates the master file later; transaction processing updates it immediately with record locking; real-time processing uses a feedback loop from sensors

Check digits and Modulus 11

Long code numbers such as barcodes, book numbers and account numbers are easy to mistype or misread, so many include a check digit: an extra digit calculated from the others and added to the end. When the number is entered or scanned, the computer repeats the calculation and compares its result with the check digit that was entered. If they disagree, the number is rejected. The Modulus 11 method gives each digit a weighting, starting at 2 for the rightmost digit and rising by one for each position to the left. Each digit is multiplied by its weighting and the products are added to give a checksum. The checksum is divided by 11 and the remainder found. If the remainder is 0 the check digit is 0; if it is 1 the check digit is X, which stands for 10; otherwise the check digit is 11 minus the remainder. Because a check digit can be X, numbers like this are stored as strings. Modulus 11 detects all single-digit errors and all transpositions of two digits, which makes it a powerful validation check, but it only shows that a number is internally consistent, not that it is the correct number.

Calculating and checking a check digit

Take the base number 3604. With four digits the weightings are 5, 4, 3 and 2. Multiplying gives 3 × 5 = 15, 6 × 4 = 24, 0 × 3 = 0 and 4 × 2 = 8, so the checksum is 47. Dividing 47 by 11 gives 4 remainder 3, and 11 − 3 = 8, so the full number is 36048. To check a number that already includes its check digit, give the check digit a weighting of 1 and repeat the calculation: 15 + 24 + 0 + 8 + 8 = 55, and 55 divided by 11 leaves remainder 0, so the number is accepted. If a clerk swaps the first two digits and types 63048, the products become 30, 12, 0, 8 and 8, totalling 58, which leaves a remainder of 3, so the error is detected. When the remainder is 1 the check digit is X; for example 1205 gives a checksum of 23, remainder 1, so the full number is 1205X, and when checking, X counts as 10. Always set the working out in a table of weightings, digits and products.

Batch, control and hash totals

When many paper documents, such as fee forms or timesheets, are typed in as a batch, normal validation of each field cannot show that every document was entered exactly once. Batch totals solve this. A batch total is simply a count of the documents, made by hand before entry and compared with the computer's count afterwards. A control total is the sum of a meaningful field on every document, such as the total fees paid. A hash total is the sum of a field that has no meaning when added up, such as member ID numbers. In each case the manual total is compared with the total calculated by the computer, and any difference shows a mistake. A batch total cannot detect one document missed and another entered twice, because the count is unchanged. A control total may also fail if the two documents happen to have the same value in that field. A hash total of unique IDs is much harder to fool, and when exactly one document is missing, the difference between the totals is that document's ID. The chosen field must vary between documents.

Data capture

Data capture is the process of getting data into a computer system. Manual methods, such as keying data from paper forms, are cheap and flexible but slow and prone to transcription and transposition errors, so validation, verification and batch totals are essential. Online forms capture data at source, so there is no re-keying, and validate each field as it is entered, but users need a device and an internet connection. Automated methods are faster and more accurate for high volumes. Optical mark recognition reads marks in fixed positions on forms such as multiple-choice answer sheets. Barcode and QR code scanners read product and parcel codes, and the check digit confirms the scan was correct. Chip and contactless card readers capture account details without typing. Sensors and data loggers record readings such as temperature or water level continuously and unattended. Automated methods need specialist equipment, and each only works when the data is in the expected form, so a choice of method depends on volume, accuracy needed, cost and the kind of data being captured.

Batch processing

In batch processing, transactions are collected over a period into a transaction file and then processed together at a later time, often overnight, with no user interaction once the job has started. The transaction file is validated, using checks such as batch and control totals, and sorted into the same key order as the master file. The program then reads both files in sequence and produces an updated master file along with outputs such as bills, payslips or re-order lists. Typical applications are payroll, utility and credit card billing, end-of-day stock re-ordering and the overnight processing of bank transactions. Batch processing makes efficient use of computer time because the same operation is repeated on many records, and it can be scheduled when the system is otherwise quiet. Its main drawback is that the master file is only up to date immediately after each run, so it is unsuitable where people need current information, such as available seats. Errors in the data may also only be discovered after the run has finished.

Real-time transaction and control processing

In real-time processing each transaction or input is dealt with as soon as it happens, so the master file is always up to date. It takes two forms. Real-time transaction processing, often called online processing, handles each transaction while the user waits a few seconds; it is used for bookings, cash machines, online shopping and anything where the next user must see the effect of the previous transaction, such as a seat being sold or a password being changed. Because many users may try to change the same record at the same moment, these systems use record locking: while one user's transaction is updating a record, other users cannot change it, which prevents lost updates and double-booking. They need reliable networks and servers that are available at all times. Real-time control goes further: sensor input must be processed within a guaranteed, very short time, so that the output can influence the next input. Anti-lock brakes, traffic lights that respond to queues and heart monitors are examples. They use feedback and must be extremely reliable, often with duplicated hardware, because a late response can be dangerous.

Multitasking, multi-user and multithreading

Modern systems process many jobs at once, and the operating system manages how they share the hardware. Multitasking allows one user to run several programs apparently at the same time: the processor switches rapidly between processes, giving each a short time slice, so that music can play while a document is edited. A multi-user system lets many users share the same computer's resources at the same time, each with their own session, files and permissions; a school file server or a bank's central system are examples, and the operating system must keep users' data separate and secure. Multithreading splits a single program into threads, smaller units of execution that share the program's memory and can run concurrently, so a web browser can download images in one thread while responding to scrolling in another. On a multicore processor threads can genuinely run in parallel, improving performance. In transaction processing systems these ideas combine: many users, each served by threads, update shared records, which is why record locking is required.

T Key terms from the mark schemes

Check digit
An extra digit calculated from the other digits of a code number and added to the end, so the computer can detect entry or reading errors by recalculating it.

Batch total
A validation check in which the number of documents in a batch is counted by hand and compared with the number entered by the computer.

Hash total
A batch total formed by adding up a field whose total has no meaning, such as ID numbers, and comparing it with the computer total.

Real-time transaction processing
Real-time processing in which each transaction is processed as it occurs while the user waits, so the master file is always up to date, e.g. bookings and cash machines.
`Credited: Online (transaction) processing`

Record locking
Preventing other users from changing a record while one user is updating it, to avoid lost updates in multi-user systems.

Multithreading
Running several threads, each a separate path of execution, within one process so they share its memory and code.

Modulus 11
A check digit method in which each digit is multiplied by a weighting, the products are added, and the check digit is found from the remainder after dividing by 11 (0 gives 0, 1 gives X, otherwise 11 minus the remainder).

Control total
A batch total formed by adding up a meaningful field on every document, such as total hours worked, and comparing it with the computer total.

Batch processing
Processing in which transactions are collected over a period in a transaction file and processed together as one job at a later time, without user intervention.

Real-time processing
Processing in which each transaction or input is dealt with as soon as it occurs, so the master file is always up to date; real-time control systems also guarantee a response fast enough to affect the next input.

Multitasking
The apparent running of several programs at once by rapidly switching the processor between them.

Multi-user
A system that allows many users to share the same computer's resources at the same time, each with their own session and permissions.

C C# example

`Modulus 11 in C#`

```
string number = "3604";
int weight = number.Length + 1;
int total = 0;
foreach (char c in number)
{
    total += (c - '0') * weight;
    weight--;
}
int remainder = total % 11;
string check;
if (remainder == 0) check = "0";
else if (remainder == 1) check = "X";
else check = (11 - remainder).ToString();
Console.WriteLine(number + check);   // 36048
```

Exam focus

Check digit questions are usually Calculate questions worth three to four marks: show the weightings, the products, the checksum, the remainder and the final check digit, and remember the rules for remainders of 0 and 1. If asked to show an error is detected, recalculate including the check digit with weighting 1 and state that the remainder is not zero. Mark schemes for batch totals credit a clear definition of each type and an applied example using a field from the scenario: count of documents for a batch total, a meaningful sum for a control total and a meaningless sum, such as IDs, for a hash total. Explaining why a hash total catches errors a batch total misses earns extra credit. For processing methods, name the method and justify it from the scenario: how quickly results are needed, whether the master file must always be current, and the volume of data. A booking system or cash machine is real-time transaction (online) processing because the master file is updated at once; anti-lock brakes are real-time control. Explain record locking in terms of preventing double-booking or lost updates.

? Check yourself

1. State the purpose of a check digit. [1]
2. A library in Strabane issues borrower numbers that end in a Modulus 11 check digit. Work out the check digit for the base number 2731, showing your working. [3]
3. A library card number is stored as 1250X. Use the Modulus 11 method to show that this number has been entered incorrectly. [3]

Answers: 1. To detect errors when a code number is entered or scanned, e.g. transposition/transcription errors (1) 2. Weightings 5, 4, 3, 2; products 10, 28, 9, 2 (1) / checksum 49; 49 ÷ 11 = 4 remainder 5 (1) / check digit 11 − 5 = 6, full number 27316 (1) 3. Weightings 5, 4, 3, 2, 1 with X = 10 (1) / products 5 + 8 + 15 + 0 + 10 = 38 (1) / 38 ÷ 11 = 3 remainder 5; remainder is not 0, so the number is invalid (1). Alternative: check digit for 1250 is 28 MOD 11 = 6, 11 − 6 = 5, not X.

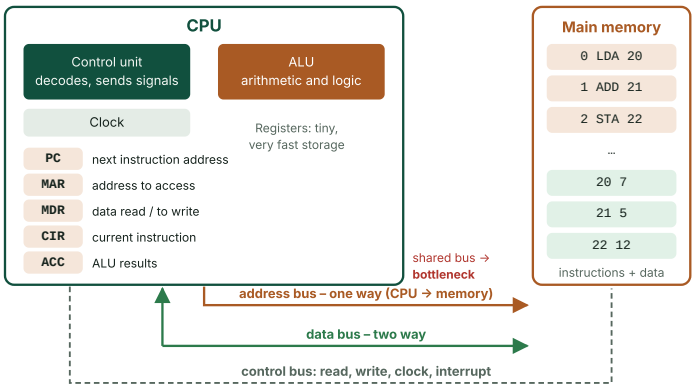
Internal components: CPU, registers, buses, I/O controllers and the Von Neumann architecture

FD08 FACT FILE

Specification: students should be able to

- Describe the internal components of a computer system: processor, clock, main memory, buses, I/O controllers, registers and ports
- Describe the tasks performed by the control unit, the arithmetic logic unit (ALU) and the registers
- Describe the purpose of the program counter (PC), memory address register (MAR), memory data register (MDR), current instruction register (CIR) and accumulator (ACC)
- Describe the function and direction of the address, data and control buses
- Calculate the maximum addressable memory, or an unknown bus width, from the address bus width, data bus width and memory size
- Explain why peripherals are connected through I/O controllers and ports
- Describe the Von Neumann (stored-program) architecture and its main limitation

Key words: central processing unit (CPU), control unit (CU), arithmetic logic unit (ALU), register, program counter (PC), memory address register (MAR), memory data register (MDR), current instruction register (CIR), accumulator (ACC), system bus, address bus, data bus, control bus, bus width, I/O controller, port, motherboard, Von Neumann architecture



The Von Neumann architecture: a CPU containing the control unit, ALU, clock and registers, connected to a single main memory holding instructions and data by the address, data and control buses

How a computer is organised

Almost every general-purpose computer follows the stored-program design set out by John von Neumann in 1945. Program instructions and the data they work on are both held, as binary, in the same main memory. A single processor fetches instructions from that memory one at a time and carries them out. Input devices supply data, output devices present results, and secondary storage keeps programs and files when the power is switched off. All of these parts are linked by the system bus, a set of parallel connections divided into address, data and control lines. Physically, the main components sit on the motherboard, a large printed circuit board with a socket for the processor, slots for memory modules, expansion slots for extra cards such as graphics or network cards, and connectors for drives and external ports. Because instructions are just data held in memory, the same computer can switch from a spreadsheet to a game simply by loading different instructions, which is what makes it general purpose. The cost of this flexibility is that instructions and data must take turns on the same bus, a limit known as the Von Neumann bottleneck.

The control unit, ALU and clock

The central processing unit is made up of the control unit, the arithmetic logic unit and a set of registers, kept in step by a clock signal. The control unit has three main jobs. It manages the fetch–execute cycle, fetching each instruction from memory, decoding it and directing its execution, calling on the ALU whenever a calculation is needed. It sends control signals that coordinate and monitor the rest of the hardware, so that the commands issued by software are carried out in the correct order. It also manages the flow of data into and out of the processor, including input and output. The ALU does the actual processing. It takes values from registers, performs whichever operation the control unit has selected and stores the result in a register, usually the accumulator. Its operations are arithmetic, such as addition and subtraction, and logical, such as AND, OR and NOT, together with comparisons like greater than or equal to, which are what allow programs to make decisions and repeat. The clock produces a regular stream of electrical pulses, and every step the processor takes is timed to them, which is why clock speed is so closely linked to performance.

Registers and what they hold

A register is a single, very small storage location built into the processor. Registers are the fastest storage in the whole computer, and data must be placed in a register before the ALU can work on it: to add two numbers, each is loaded into a register, the ALU adds them and the answer is written back to a register. A register can hold a data value, an instruction or the address of a memory location. General-purpose registers are used freely by programs, while special-purpose registers each have a fixed role in the fetch–execute cycle. The program counter holds the address of the next instruction to be fetched and is incremented after each fetch. The memory address register holds the address of the location about to be read or written. The memory data register is a two-way buffer between processor and memory, holding whatever has just been read or is waiting to be written. The current instruction register holds the instruction being decoded and executed, and the accumulator holds the results of ALU operations. Programmers working in a high-level language such as C# never handle registers themselves; the compiler produces machine code that does.

The system bus

A bus is a group of parallel wires, or tracks on the motherboard, that carries electrical signals between components. Each wire carries one bit at a time, so the number of wires, called the bus width, decides how many bits travel together. The system bus has three parts. The address bus identifies which memory location or I/O device the processor wants to use. It is unidirectional, because only the processor chooses which location to access, so addresses travel outwards from the CPU. The data bus carries the data itself, including instructions, between the processor, memory and I/O devices. It is bidirectional because data is both read into and written out of the processor. The control bus carries command and timing signals such as memory read, memory write, the clock pulse, interrupt requests and bus requests. Each individual control line carries one kind of signal in one direction, but the control bus as a whole is bidirectional because some signals go out from the CPU while others, such as an interrupt from a keyboard or a job-finished message from a printer, come back in. A wider data bus moves more bits per transfer, which improves performance.

Calculating addressable memory

The width of the address bus fixes how many different locations the processor can identify. With n address lines there are 2^n different binary patterns, so 2^n locations numbered from 0 to $2^n - 1$, and each extra line doubles the total. Each location holds as many bits as the data bus carries in one transfer, so the maximum memory in bits is 2^n multiplied by the data bus width. To convert, divide by 8 (2^3) for bytes and then by 2^{10}, 2^{20} or 2^{30} for kilobytes, megabytes or gigabytes, since this course measures memory in binary multiples. Writing every value as a power of two keeps the arithmetic simple, because multiplying means adding indices and dividing means subtracting them. For example, a 24-bit address bus with a 16-bit data bus gives $2^{24} \times 2^4 = 2^{28}$ bits, which is 2^{25} bytes or 32 MB. The same formula can be rearranged to find a missing bus width when the memory size is known. In practice few computers are fitted with all the memory their address bus allows: memory costs money, most software does not need that much, and some addresses are reserved for I/O ports rather than RAM.

I/O controllers and ports

Everything outside the processor, main memory and system bus is a peripheral: an input device, an output device, or a device that does both, such as an external drive. Peripherals work at very different speeds and voltages and use different command formats, so it would be impractical to design a processor able to drive every one directly. Instead each device is connected through an I/O controller, an electronic circuit that sits between the system bus and the device. It contains an interface to the system bus, electronics that exchange signals with the device, and an I/O port through which data passes. The controller converts between the protocol the device uses and the one the computer uses, changes voltage levels to suit the computer, buffers data so that a fast processor is not held up by a slow device, and converts data formats, for example turning a 16-bit value from a device into the 32-bit form the computer expects. Controllers may be built into the motherboard or added on expansion cards. A port is the physical socket where a cable connects. USB has largely replaced the many dedicated ports of older PCs, carrying both data and power through one standard connector.

T Key terms from the mark schemes

Arithmetic logic unit (ALU)
The part of the CPU that performs arithmetic operations, logical operations and comparisons on data held in registers.

Control unit (CU)
The part of the CPU that fetches and decodes instructions and sends control signals to coordinate the processor, memory and I/O.
`Credited: Coordinates the actions of the computer / manages the fetch–execute cycle`

Register
A single, very fast storage location inside the CPU that holds data, an instruction or an address during processing.

Program counter (PC)
A register that holds the address of the next instruction to be fetched; it is incremented after each fetch.

Memory address register (MAR)
A register that holds the address of the memory location currently being read from or written to.

Memory data register (MDR)
A register that holds data or an instruction just read from memory, or data waiting to be written to memory.
`Credited: Memory buffer register (MBR)`

Current instruction register (CIR)
A register that holds the instruction currently being decoded and executed.

Accumulator (ACC)
A register that holds the results of calculations and logical operations performed by the ALU.

Address bus
The unidirectional part of the system bus that carries the address of a memory location or I/O device from the CPU.

Data bus
The bidirectional part of the system bus that carries data and instructions between the CPU, memory and I/O devices.

I/O controller
An electronic circuit between the system bus and a peripheral that converts protocols, voltages and data formats and buffers data.
`Credited: Device controller`

Von Neumann architecture
A computer design in which instructions and data are stored together in the same memory and accessed by a single processor over a shared bus.
`Credited: Stored-program concept`

C C# example

`Checking the calculation in C#`

```
int addressBits = 30;
int dataBits = 64;

long locations = 1L << addressBits;            // 2^30 locations
long totalBytes = locations * (dataBits / 8);  // each location holds 8 bytes
double gigabytes = totalBytes / Math.Pow(2, 30);

Console.WriteLine($"Locations: {locations}");
Console.WriteLine($"Maximum memory: {gigabytes} GB");
```

Exam focus

'State' or 'name' questions earn one mark per correct component, so use full names such as memory address register before abbreviating. 'Describe the purpose of' needs what the item holds or does plus a detail: the program counter holds the address of the next instruction and is incremented after each fetch. Questions on the control unit often carry three marks, one for each task, and bus questions usually give one mark per bus for what it carries, with credit for the correct direction. In calculations, show every step in powers of two; method marks are given even if the final figure slips, and a bare answer risks losing them all. Keep bits and bytes separate and divide by 8 at the right point. Common errors include confusing the MAR, which always holds an address, with the MDR, which holds data or an instruction; saying the control bus carries addresses; and calling the address bus two-way. For I/O controllers, examiners credit reasons linked to differences in speed, voltage and protocol between devices. When describing the Von Neumann architecture, mention shared memory for instructions and data, the single shared bus and the resulting bottleneck.

? Check yourself

1. State two tasks carried out by the control unit. [2]
2. Describe the purpose of the memory data register (MDR). [2]
3. Explain why the address bus is unidirectional but the data bus is bidirectional. [2]

Answers: 1. Any two from: fetches instructions from memory, decodes and executes them (fetch–execute cycle), using the ALU as needed (1) / sends control signals to coordinate and monitor hardware so commands are carried out (1) / controls the input and output of data / movement of data into and out of the processor (1) 2. Holds data/an instruction that has just been read (fetched) from memory (1) / or data that is waiting to be written to memory (1) – acts as a buffer between the CPU and memory 3. Only the CPU sends addresses / memory and I/O never need to send an address back, so the address bus only carries signals from CPU to memory/I/O (1) / data must travel both to and from the CPU (reads and writes), so the data bus is two-way (1)

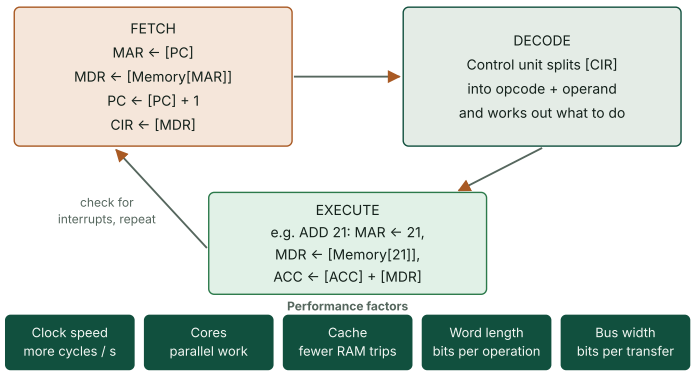
The fetch–decode–execute cycle, pipelining and processor performance

FD09 FACT FILE

Specification: students should be able to

- Describe the fetch–execute cycle, including the fetch, decode and execute stages and the roles of the registers and buses
- Describe how pipelining overlaps the stages of the cycle, and calculate the time units needed with and without pipelining
- Evaluate the factors that influence the speed of processing: processor type (word size, number of cores) and clock speed
- Explain how cache size, bus width and the amount of RAM also affect performance
- Explain why clock speed alone is not a good measure of performance, and describe benchmark measures such as MIPS and FLOPS
- Interpret a computer specification, such as '64-bit 3.2 GHz quad-core'

Key words: fetch–decode–execute cycle, fetch, decode, execute, instruction set, opcode, operand, pipelining, clock speed, hertz (Hz), multi-core processor, word size, 32-bit / 64-bit processor, cache, bus width, benchmark, MIPS, FLOPS



The fetch-decode-execute cycle shown as a loop with the register transfers at each stage, and the factors that affect processor performance

The cycle in overview

Every program, from a C# console app to a web browser, is run by the processor repeating the same three-stage routine for each instruction: fetch, decode and execute. Before the cycle can begin, the operating system copies the program's instructions and data from secondary storage into main memory, because the processor can only fetch directly from RAM. In the fetch stage the next instruction is copied from memory into the processor. In the decode stage the control unit works out what the instruction means. In the execute stage the instruction is carried out, which might involve a calculation by the ALU, reading or writing memory, or changing which instruction comes next. The cycle then repeats, continuously, from power-on until shutdown, and a modern processor completes billions of these cycles every second. The routine is often called the fetch–execute cycle, but it has three distinct stages. The speed at which a processor gets through the cycle depends on its clock speed, how many cores it has, its word size, its cache and how quickly memory can supply it, while pipelining lets the stages of different instructions overlap so more work is completed in the same time.

Fetch in detail

The fetch stage uses the special-purpose registers and all three buses. The program counter always holds the address of the next instruction to be fetched; when a program starts it holds the address of the first instruction. That address is copied from the program counter into the memory address register. The program counter is then incremented so that it already points to the instruction that follows, ready for the next cycle. The memory address register places the address on the address bus, and the control unit sends a memory-read signal along the control bus. Main memory responds by placing the contents of that location on the data bus, and they arrive in the memory data register. Finally, the instruction is copied from the memory data register into the current instruction register, where it will be decoded. Textbooks differ over exactly when the program counter is incremented, but it must happen during the fetch and after its contents have been copied to the memory address register. In an exam answer, keep the steps in order and name each register and bus in full, because a sequence that misses the increment or muddles the registers will lose marks.

Decode and execute

In the decode stage the control unit interprets the instruction held in the current instruction register. A machine-code instruction has two parts: the opcode, which identifies the operation, such as load, add, store or jump, and the operand, which is either a value to be used or the address where the value is found. Each processor family has its own instruction set, the complete list of opcodes it recognises, which is why a program compiled for one family cannot run on a different one. Having decoded the instruction, the control unit sets up the registers and circuits needed. In the execute stage it sends the control signals that carry the instruction out. If the operand is an address, that address is placed in the memory address register so that data can be read into, or written out of, the memory data register. Arithmetic and logical operations are performed by the ALU, with the result held in the accumulator. A jump or branch instruction works differently: it replaces the contents of the program counter, so the next fetch comes from a new part of the program. This is how selection and loops in high-level code are carried out at machine level.

Pipelining

In a simple processor each instruction is fully fetched, decoded and executed before the next fetch starts, so with three stages of one time unit each, n instructions take 3n time units. Pipelining overlaps the stages of different instructions. As soon as the first instruction moves on to be decoded, the second is fetched; when the first is executed, the second is decoded and the third is fetched. Once the pipeline is full, one instruction completes in every time unit, so n instructions need only n + 2 units. Four instructions therefore take six units instead of twelve, and three take five instead of nine. Each individual instruction still takes three units, but throughput, the number of instructions completed per second, rises without any increase in clock speed. The method works best when instructions follow each other in order. A branch or jump means the instructions already fetched and decoded behind it may be the wrong ones, so the pipeline has to be emptied and refilled, wasting some time units. Modern processors use pipelines with many more than three stages, along with techniques to predict which way a branch will go, but the principle is the same.

Clock speed and processor type

The clock produces a regular series of pulses, and each step of the cycle takes one or more of them. Clock speed is the number of cycles per second, measured in hertz, so a 3.2 GHz processor completes 3.2×10^9 cycles every second. Raising the clock speed lets more instructions be processed per second, but it also increases power consumption and heat; a chip that gets too hot can be damaged, so faster processors need better cooling and drain laptop batteries sooner. Processor type matters as much. Word size is the number of bits the processor treats as a single unit, matching the size of its general registers. A 64-bit processor handles larger values in one operation and, unlike 32-bit addressing, which is limited to 2^{32} bytes or 4 GB, can make use of far more memory. A 64-bit processor will run 32-bit software, but programs built for 64-bit cannot run on a 32-bit processor, and a 32-bit operating system cannot use a 64-bit processor fully. A multi-core processor contains two or more independent cores on one chip, each able to run its own stream of instructions, so several threads or programs run at once, provided the software has been written to divide its work.

Other factors and benchmarks

Several other features affect how quickly a computer processes data. Cache memory holds frequently used instructions and data beside the processor; a larger cache means fewer slow trips to RAM, although cache is costly and the benefit levels off. A wider data bus moves more bits in each transfer. More RAM means the operating system has to swap data out to slow secondary storage less often. Because so many factors interact, comparing processors by clock speed alone is misleading: two chips running at the same speed can differ greatly in how many instructions they complete per cycle, how deep their pipeline is, how many cores they have and how much cache they carry. Upgrading one component can also simply move the bottleneck elsewhere, for example a faster processor left waiting for a slow hard disk. A fairer comparison uses a benchmark, a standard set of tasks run on each machine with the results timed. Processor benchmarks are often reported in MIPS (millions of instructions executed each second) or in FLOPS (the number of floating-point calculations completed each second), which matters for science, graphics and machine learning. These figures describe the processor only and ignore storage and network speed.

Key terms from the mark schemes

Fetch–decode–execute cycle
The continuous process by which the CPU fetches an instruction from memory, decodes it and carries it out.
Credited: Fetch–execute cycle / instruction cycle

Opcode
The part of a machine-code instruction that specifies the operation to be performed.
Credited: Operation code

Operand
The part of a machine-code instruction that holds the data to be used or the address where the data is stored.

Instruction set
The complete set of machine-code instructions that a particular processor is designed to recognise and execute.

Pipelining
Overlapping the fetch, decode and execute stages of successive instructions so that several instructions are being processed at once.

Clock speed
The number of clock cycles the processor carries out per second, measured in hertz (e.g. 3.5 GHz).
Credited: Clock rate

Core
An independent processing unit within a CPU, able to fetch, decode and execute its own instructions.

Multi-core processor
A single processor chip containing two or more independent cores that can execute instructions at the same time.
Credited: Dual-core / quad-core

Word size
The number of bits a processor can handle as a single unit in one operation, usually equal to the size of its registers.
Credited: Word length

Cache
Small, very fast memory close to the CPU that stores frequently used instructions and data to reduce slower accesses to RAM.

Benchmark
A standard set of tasks run on different computers so that their performance can be measured and compared fairly.

FLOPS
Floating-point operations per second: a measure of a processor's speed at arithmetic on real numbers.
Credited: MIPS (millions of instructions per second) for general instructions

C C# example

Simulating the cycle in C#

```
int[] memory = { 104, 205, 306, 0, 7, 5, 0 };
int pc = 0, acc = 0;
while (true)
{
    int mar = pc;                 // fetch: MAR <- PC
    int mdr = memory[mar];        // MDR <- memory[MAR]
    pc++;                         // PC <- PC + 1
    int cir = mdr;                // CIR <- MDR
    int opcode = cir / 100, operand = cir % 100;   // decode
    if (opcode == 0) break;                        // HLT
    if (opcode == 1) acc = memory[operand];        // LDA
    else if (opcode == 2) acc += memory[operand];  // ADD
    else if (opcode == 3) memory[operand] = acc;   // STA
    Console.WriteLine($"PC={pc} CIR={cir} ACC={acc}");
}
Console.WriteLine($"Address 6 holds {memory[6]}");
```

Exam focus

Fetch–execute questions are usually 'describe' or 'explain' items worth three to six marks, with one mark for each correct, correctly ordered step. Name registers in full and say what moves where, for example 'the contents of the program counter are copied to the memory address register', rather than vague phrases like 'the PC goes to memory'. Remember to include the increment of the program counter, the read signal on the control bus and the copy from the memory data register to the current instruction register. For decode, mention opcode and operand; for execute, mention the ALU and the accumulator. Pipelining questions may ask for a diagram: draw time units down the side and instructions across, with stages staggered by one unit, and state the total. Performance questions reward a reason linked to an effect: 'more cores allow several threads to be executed at the same time' rather than 'more cores is faster'. When asked why clock speed alone is a poor measure, explain that processor architecture, cores, cache and pipelining differ. Clock speeds use decimal prefixes (1 GHz = 10^9 Hz); if a question sets a different convention, follow it and show your working.

? Check yourself

1. State the three stages of the fetch–execute cycle. [1]
2. Describe the fetch stage of the fetch–decode–execute cycle, naming the registers and buses involved. [6]
3. Explain what happens during the decode stage, referring to the opcode and operand. [3]

Answers: 1. Fetch, decode, execute (all three for 1 mark) 2. The address in the program counter (PC) is copied into the memory address register (MAR) (1) / the PC is incremented to point to the next instruction (1) / the address held in the MAR is put onto the address bus (1) / a read signal is sent on the control bus (1) / the instruction at that address is sent along the data bus into the memory data register (MDR) (1) / the instruction is copied from the MDR into the current instruction register (CIR) (1) 3. The control unit interprets the instruction in the CIR (1) / it splits it into the opcode, which identifies the operation to perform (1) / and the operand, which is the data or the address of the data (1) / (accept: prepares registers for execution)

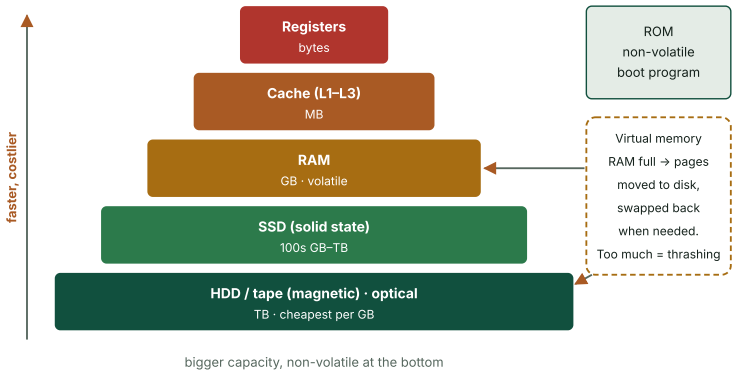
Main memory, cache and secondary storage

FD10 FACT FILE

Specification: students should be able to

- Describe main memory, distinguishing between RAM and ROM, and explain the uses of each
- Describe the purpose of cache memory, how a cache controller uses it, and the levels of cache
- Explain how virtual memory is used and its effect on performance
- Explain the need for secondary storage and the factors that influence the choice of storage device
- Describe magnetic, optical and flash (solid-state) secondary storage media, including how data is stored and accessed
- Evaluate the use of secondary storage media for common applications

Key words: main memory, random access memory (RAM), read-only memory (ROM), volatile, non-volatile, cache memory, cache controller, virtual memory, disk thrashing, secondary storage, magnetic storage, tracks and sectors, serial access, direct access, optical storage, flash memory, solid-state drive (SSD)



The memory hierarchy from registers to magnetic and optical storage: faster and more expensive at the top, larger at the bottom, with virtual memory linking RAM and secondary storage

Memory and storage in overview

A computer uses several kinds of storage, arranged as a hierarchy. At the top are the registers and cache, which are tiny, extremely fast and costly per byte. Next is main memory, the store the processor can address directly, which holds the programs and data in use. At the bottom is secondary storage, which is much slower but far larger and cheaper, and which keeps its contents when the power is off. The design aims to give the processor the speed of the fast levels while providing the capacity of the slow ones, by moving data up the hierarchy just before it is needed. When you open a file, it is copied from secondary storage into RAM, and the parts the processor is using are copied again into cache. Main memory consists of RAM, which is volatile and can be read and written, and ROM, which is non-volatile and holds fixed programs. Secondary storage comes in three main technologies: magnetic, such as hard disks and tape; optical, such as CDs, DVDs and Blu-ray discs; and flash, such as SSDs, USB sticks and memory cards. Choosing the right device means weighing speed, capacity, durability, portability, reliability and cost against what the user needs.

RAM and ROM

Random access memory holds the operating system, applications and data that are currently in use. The processor can read from it and write to it, and any location can be reached directly in the same time. It is volatile, so everything in it is lost when the power is switched off, which is why programs must be loaded into RAM each time they are used and why work must be saved to secondary storage. RAM is measured in gigabytes, and because secondary storage is so much slower, having more RAM generally makes a computer more responsive: more programs and data can stay in memory rather than being fetched repeatedly from disk. Read-only memory is non-volatile and its contents are fixed when it is made, so the user cannot change them in normal use. On a PC it holds the firmware that runs when the computer is switched on, testing the hardware and loading the operating system from secondary storage, which has to be in ROM because RAM is empty at that point. ROM is also used in embedded systems, such as the controller in a dishwasher or central heating timer, where the program never needs to change. Many modern devices use rewritable flash memory for this firmware so that it can be updated.

Cache memory

Even though RAM is much faster than a disk, the processor can still work faster than RAM can supply it, so it would spend time waiting. Cache memory solves this by keeping copies of data and instructions in a small, very fast memory on or right next to the processor. Cache is made from a faster and more expensive type of memory than RAM, so a processor with more cache costs more. It has two jobs: it keeps instructions and data that are used often, and it reads ahead. A cache controller manages the transfer of data from RAM into the cache. When the processor needs an item, it first checks the cache. If the item is there, a cache hit, it is supplied immediately. If not, a cache miss, the item is fetched from RAM, placed in the cache and then used, and the controller may also fetch the neighbouring addresses, since program instructions usually follow one another in memory. Cache is organised in levels: level 1 is the smallest and fastest and is built into each core, level 2 is larger and slightly slower, and level 3 is larger again and usually shared by all the cores on the chip.

Virtual memory

When so many programs are open that RAM is full, the operating system can use an area of secondary storage as if it were extra RAM. This is virtual memory. Memory is managed in fixed-size blocks called pages. Pages that are not being used at the moment are copied out of RAM to the disk, freeing space for the pages that are needed now. When a page held on the disk is needed again, it is copied back into RAM, which may mean another page has to be moved out to make room. Virtual memory lets a user run more programs, or larger ones, than the physical RAM would otherwise allow, and it stops the computer simply refusing to open another program. The cost is speed. Even an SSD is far slower than RAM, and a hard disk is slower still, so every swap holds the processor up. If RAM is badly overloaded, the computer can spend more time moving pages back and forth than doing useful work, a condition called disk thrashing, in which the system becomes very sluggish and the disk activity light may stay on. The usual cure is to fit more RAM or close some programs, which reduces how often virtual memory is needed.

Magnetic and optical storage

A hard disk drive contains one or more rigid platters coated with magnetic material, mounted on a spindle inside a sealed case. Each surface is organised into concentric tracks, which are divided into sectors. The platters spin at thousands of revolutions per minute while read/write heads on an actuator arm move across them. To write, the operating system looks up its file system table for free space, the data is passed to the disk controller, the arm moves the head to the right track, and when the required sector spins beneath it, tiny regions are magnetised in one of two directions to represent 0s and 1s. Hard disks give direct access, huge capacity and a low cost per gigabyte, but their moving parts can be damaged by a knock. Magnetic tape stores data serially, so the drive must wind through the tape to reach a file. It is slow to search but very cheap for its vast capacity, so it suits archives and backups. Floppy disks used similar principles but are obsolete. Optical discs store data as marks on a reflective layer that a laser reads. CDs, DVDs and Blu-ray discs come as ROM, recordable (R) and rewritable (RW) types; they are cheap and portable but slow and easily scratched.

Flash memory and SSDs

Flash memory stores data as electrical charges held in semiconductor cells. It is non-volatile and has no moving parts, which makes it fast, quiet and resistant to being dropped or shaken. USB memory sticks and memory cards, such as the SD and microSD cards used in cameras, drones and phones, are small, light and portable. When a USB stick is plugged in, the operating system loads a suitable driver, reads the stick's file table to learn its folder structure, and then, when a file is opened, sends the required address to the device, which returns the data. A solid-state drive uses the same technology but is designed to sit inside the computer in place of a hard disk. Compared with a hard disk, an SSD gives much faster access because there is no waiting for a head to move or a platter to spin; it uses less power, which extends laptop battery life; it is smaller and lighter, suiting thin laptops and tablets; and it survives knocks better. Its disadvantages are a higher cost per gigabyte and the fact that each cell can be rewritten only a limited number of times, although modern drives spread writes evenly to prolong their life. Falling prices mean SSDs are now the normal main drive in most new laptops.

Choosing storage for an application

Every storage question comes down to matching the medium to the need. Speed matters when data must be opened instantly, as with a booking system or the drive holding the operating system, which favours an SSD. Capacity matters for large collections of video, images or records, which favour hard disks or, for archives, tape. Durability and reliability matter in harsh conditions: a camera on a racing car, a field-research tablet or a delivery driver's handheld needs flash storage with no moving parts. Portability matters when data has to travel, which favours USB sticks, memory cards and optical discs. Cost per gigabyte matters for bulk storage and backups, where tape and hard disks are cheapest. The type of access needed is also important: tape is serial, so it is a poor choice when individual files are needed often, but fine for a backup that will only be restored in an emergency. Whether data must be changed matters too: a write-once disc protects a distributed file from alteration, while a rewritable medium suits work in progress. Increasingly, organisations also use cloud storage and network drives, which add off-site protection and sharing but depend on an internet connection and on trust in the provider.

T Key terms from the mark schemes

Main memory
The memory that the processor can address directly, holding the programs and data currently in use (RAM and ROM).
`Credited: Primary memory / immediate access store`

RAM
Random access memory: volatile read/write main memory that holds the operating system, programs and data currently in use.

ROM
Read-only memory: non-volatile main memory whose contents are fixed when made, used for start-up firmware and embedded programs.

Volatile
Describes memory that loses its contents when the power is switched off.

Cache memory
Small, very fast and expensive memory on or beside the processor that holds frequently used data and instructions and data read ahead from RAM.
`Credited: CPU memory`

Cache controller
The device that transfers data from RAM into cache, predicting and pre-fetching the data the processor is likely to need next.

Virtual memory
An area of secondary storage used by the operating system as an extension of RAM when RAM is full, with pages swapped in and out.

Disk thrashing
Severe loss of performance when the computer spends more time swapping pages between RAM and virtual memory than processing.

Secondary storage
Non-volatile storage that keeps programs and data permanently when the power is off, e.g. hard disk, SSD, optical disc.
`Credited: Backing storage / auxiliary storage`

Serial access
Data must be read in order from the start until the required item is reached, as on magnetic tape.
`Credited: Sequential access`

Optical storage
Storage in which a laser writes and reads marks on the reflective surface of a disc, e.g. CD, DVD and Blu-ray.

Solid-state storage
Non-volatile storage using flash memory with no moving parts, e.g. SSDs, USB drives and memory cards.
`Credited: Flash memory`

Exam focus

Definition questions on RAM and ROM usually award one mark per characteristic, so state volatility, whether it can be written to, and what it is used for, and make the difference explicit. Avoid saying ROM stores the operating system on a PC: it stores the start-up firmware that loads the operating system. Cache answers should mention speed, location near the processor, frequently used data and reading ahead, and a diagram question expects the CPU, cache, cache controller and RAM with the hit and miss paths. Describe virtual memory as a process: RAM full, pages moved to secondary storage, swapped back when needed, slower performance. Storage scenario questions often say 'do not use the same device or reason twice', and marks come mainly from the justification, which must be linked to the scenario rather than generic. Use the factors examiners credit: speed, capacity, durability, portability, reliability and cost. 'Fast' alone earns little; 'an SSD has faster access than a hard disk because it has no moving parts' earns more. Remember that a CD-ROM drive cannot write data, that tape gives serial access, and that SSD advantages include low power, speed, robustness and small size.

? Check yourself

1. State two differences between RAM and ROM. [2]
2. Explain why the start-up instructions of a PC are stored in ROM rather than RAM. [2]
3. Using a diagram, explain how the CPU makes use of cache memory when it needs an instruction. [4]

Answers: 1. Any two from: RAM is volatile, ROM is non-volatile (1) / RAM can be read and written, ROM can only be read (in normal use) (1) / RAM holds programs and data currently in use, ROM holds fixed start-up programs/firmware (1) / RAM is usually much larger than ROM (1) 2. RAM is volatile so it is empty when the computer is switched on (1) / ROM is non-volatile so the start-up instructions are available immediately to load the operating system (1) 3. Diagram showing CPU, cache, cache controller and RAM (1) / CPU first checks the cache for the instruction (1) / if present (cache hit), it is used directly, having been pre-fetched by the cache controller (1) / if not (miss), it is fetched from RAM, placed in cache and then processed; controller may read further data ahead (1)

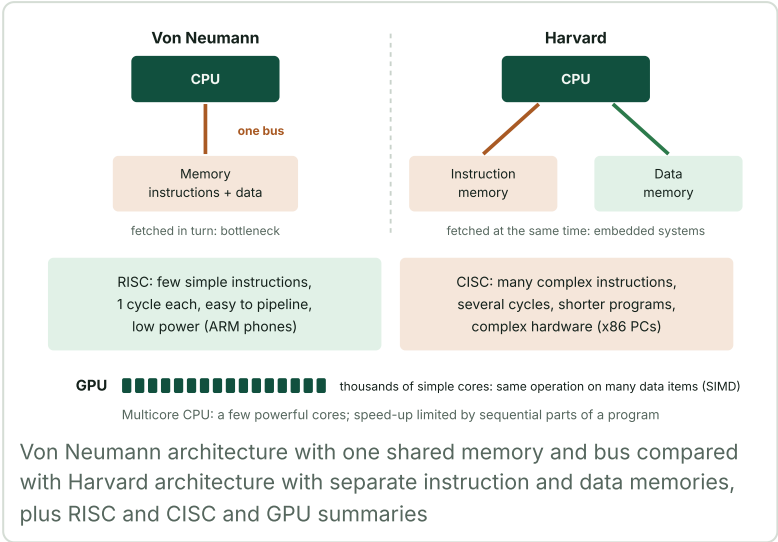
Alternative architectures: parallel, multicore, Harvard, RISC and CISC, GPUs

FD11 FACT FILE

Specification: students should be able to

- Describe parallel processing and multi-core processors, and explain their benefits and limitations
- Compare the Harvard and Von Neumann architectures and identify where each is used
- Compare reduced instruction set computers (RISC) and complex instruction set computers (CISC)
- Describe the use of graphics processing units (GPUs) and other co-processors for graphics and general-purpose parallel processing
- Evaluate the choice of processor architecture for a given application

Key words: parallel processing, multi-core processor, thread, speed-up, Von Neumann bottleneck, Harvard architecture, embedded system, microcontroller, instruction set, reduced instruction set computer (RISC), complex instruction set computer (CISC), pipelining, graphics processing unit (GPU), SIMD, co-processor



Von Neumann architecture with one shared memory and bus compared with Harvard architecture with separate instruction and data memories, plus RISC and CISC and GPU summaries

Beyond the basic design

The simple Von Neumann machine fetches and executes one instruction at a time, from one memory, over one shared set of buses. For decades performance grew mainly by raising the clock speed, but faster clocks use more power and produce more heat, and beyond a certain point a chip cannot be cooled cheaply or run from a battery for long. Designers therefore look for other ways to get more work done every second, and the alternative architectures in this lesson each tackle a different limit. Parallel processing, multi-core processors and graphics processing units carry out many operations at the same time. The Harvard architecture removes the bottleneck of a shared memory bus by giving instructions and data separate paths. Reduced instruction set computers simplify the instructions themselves so that each one finishes quickly and the pipeline keeps flowing, using less power. Real devices mix these ideas: a smartphone chip typically combines several RISC cores, a GPU and other co-processors, with separate instruction and data caches inside each core. Understanding the strengths and limits of each approach lets you explain why a games console, an office PC, a smartwatch and a washing machine contain such different processors, and choose a suitable design for a given job.

Parallel and multi-core processing

Parallel processing means using two or more processors or cores to work on different parts of a task at the same time. A multi-core processor places several complete cores on one chip, each with its own control unit, ALU, registers and level 1 cache, usually sharing a larger level 3 cache. The operating system schedules programs, or threads within a program, onto the cores so they run simultaneously, which improves multitasking and speeds up software written to divide its work. Gains depend on how much of a task can be split. Any part that must run in order, because each step needs the result of the one before, still runs on a single core, so it sets a floor on the total time. If a job takes 100 seconds and 20 of those seconds are sequential, no number of cores can bring it below 20 seconds. On top of this, dividing the work, keeping the cores coordinated and combining results all take time, and cores compete for shared cache, memory and bus. That is why a quad-core processor rarely runs a program four times faster. Parallel systems also need operating systems and programs that are written to use threads, which makes them harder to develop and test.

Harvard and Von Neumann

In the Von Neumann architecture, instructions and data share one memory and one set of buses, so they must be fetched one after the other, and the processor can be left waiting: the Von Neumann bottleneck. Its advantages are simplicity, lower cost and flexibility, since any part of memory can hold either a program or data, which suits general-purpose computers that run many different programs. The Harvard architecture uses physically separate memories and buses for instructions and for data. The processor can therefore fetch the next instruction while reading or writing data, improving speed and making timing predictable. The two memories can also be built differently: the instruction memory can be non-volatile flash of one word length, and the data memory a small amount of RAM of another. The drawbacks are extra complexity and less flexibility, because spare instruction memory cannot be used to hold data. Harvard designs are common in embedded systems, the dedicated computers inside devices such as heating controllers, car braking systems and card readers, and in digital signal processors that handle audio and video streams. Modern PCs use a hybrid: main memory follows Von Neumann, but each core has separate level 1 caches for instructions and for data.

RISC and CISC

Every processor has an instruction set, the complete set of machine-code operations it can carry out. A complex instruction set computer provides a large number of instructions, some of which perform several low-level steps in one go, such as reading two values from memory, multiplying them and storing the answer. Instructions vary in length and may take several clock cycles. CISC designs date from a time when memory was expensive and compilers were simple, so packing more work into each instruction kept programs small. A reduced instruction set computer provides a smaller set of simple, fixed-length instructions, most completing in a single clock cycle, with only load and store instructions accessing memory and everything else working on registers. A RISC program needs more instructions for the same task, and the compiler does more of the work, but the hardware is simpler. Simpler circuits use fewer transistors, consume less power and produce less heat, and uniform instructions keep a pipeline full. This is why ARM processors, which are RISC, dominate phones and tablets and now appear in many laptops, while x86 processors from Intel and AMD, which are CISC, remain common in desktop PCs largely because of compatibility with existing software. Internally, modern x86 chips break complex instructions into simpler RISC-like steps.

GPUs and co-processors

A co-processor is an additional processor that takes over a particular type of work from the main CPU. The most important is the graphics processing unit. A CPU has a small number of powerful cores with large caches and complex control logic, designed to handle varied, sequential tasks full of decisions. A GPU instead has hundreds or thousands of simpler cores that apply the same instruction to many items of data at once, an approach called SIMD, single instruction, multiple data. Graphics fits this perfectly: to draw a frame, the same lighting or colour calculation must be carried out for millions of pixels, and each pixel can be worked out independently. The same strength makes GPUs valuable outside graphics. Training machine-learning models involves enormous numbers of identical multiplications on large arrays of numbers, and video encoding, cryptography, weather forecasting and scientific simulations have similar patterns. GPU performance is often quoted in FLOPS, floating-point operations per second. GPUs are poor at sequential, branching work, so they cannot replace the CPU, which still runs the operating system and coordinates everything. Phone chips also contain other co-processors, such as image signal processors and neural engines, each optimised for one kind of task while using little power.

Choosing an architecture

No single design is best for everything; the right choice depends on what the device must do, how much power it can use, how much it can cost and what software it must run. A fitness tracker or smartwatch needs to run for days on a tiny battery, so a low-power RISC microcontroller, often using a Harvard-style memory arrangement, is ideal. A washing machine or heating controller runs one fixed program that must respond reliably to sensors, which suits an inexpensive Harvard microcontroller with its program in flash memory. An office PC must run a wide range of existing Windows applications, so a multi-core x86 processor with Von Neumann main memory remains the usual choice, although ARM-based laptops now offer longer battery life. A smartphone combines multi-core RISC processing with a built-in GPU and other co-processors to give smooth graphics and long battery life without a fan. Training an artificial intelligence model or running a weather simulation needs vast numbers of identical calculations, so GPUs or large many-core parallel systems are used. When evaluating, weigh speed for the actual workload, power and heat, cost, size and software compatibility, and remember that adding parallel hardware only helps when the task can be split.

T Key terms from the mark schemes

Parallel processing
The simultaneous use of two or more processors or cores to work on different parts of the same task.

Thread
A sequence of instructions within a program that can be scheduled to run independently, for example on a separate core.

Embedded system
A computer system built into a larger device to perform one dedicated function, such as a heating controller.
`Credited: Microcontroller (for the processor used)`

CISC
Complex instruction set computer: a processor with a large set of instructions, some of which carry out several low-level operations and take several cycles.

SIMD
Single instruction, multiple data: one instruction is applied to many data items at the same time.

Pipelining
Overlapping the fetch, decode and execute stages of successive instructions so that several are being processed at once.

Multi-core processor
A single chip containing two or more complete processing cores that can each execute instructions at the same time.

Harvard architecture
A computer design with separate memories and buses for instructions and data, so both can be accessed at the same time.

RISC
Reduced instruction set computer: a processor with a small set of simple, fixed-length instructions, most executed in one clock cycle.

Graphics processing unit (GPU)
A processor with many simple cores designed to perform the same operation on many data items in parallel, used for graphics and other parallel tasks.

Co-processor
An additional processor that takes over a specific type of work from the main CPU, such as graphics or sound.

C C# example

`Splitting work across threads in C#`

```
using System.Threading.Tasks;

int[] brightness = new int[8];

// Each image is processed on whichever core is free
Parallel.For(0, brightness.Length, i =>
{
    brightness[i] = (i + 1) * 10;   // stand-in for slow image processing
});

Console.WriteLine($"Cores available: {Environment.ProcessorCount}");
Console.WriteLine(string.Join(", ", brightness));
```

Exam focus

Comparison questions reward paired points that state both sides, for example 'RISC instructions usually take one clock cycle, whereas some CISC instructions take several'. A list of facts about only one side earns little. Link each feature to a consequence: fewer transistors lead to lower power use and longer battery life; separate buses let an instruction and data be fetched at the same time. For 'describe parallel processing', mention two or more processors or cores, working simultaneously, on different parts of the same task. For limitations of more cores, give a reason such as sequential parts of a program, software not written for threads, or competition for shared memory, rather than simply saying 'it does not always help'. When explaining why a GPU is used, mention many simple cores and the same operation applied to many data items in parallel, and link this to the task in the question. Harvard questions often ask for a typical use: embedded systems, microcontrollers or digital signal processors earn credit, but 'a desktop PC' does not. Avoid absolute claims such as 'RISC is always faster' or 'more cores always doubles speed'. In extended questions, apply each point to the scenario and finish with a justified recommendation.

? Check yourself

1. Describe what is meant by parallel processing. [2]
2. Explain two limitations of adding more cores to a processor. [4]
3. A data-analysis job takes 60 seconds on one core. 45 seconds of the work can be run in parallel and the rest cannot. Calculate how long the job takes on 3 cores, and state the shortest time it could ever take however many cores are added. [3]

Answers: 1. Two or more processors/cores are used at the same time (1) / to work on different parts of the same task/program, which is split into parts (1) 2. Any two explained, e.g.: some parts of a program are sequential (1) so they cannot be split and still run on one core, limiting speed-up (1) / software must be written to use multiple threads (1) otherwise extra cores are unused (1) / cores share cache, memory and bus (1) so they may wait for each other (1) / overhead of splitting work and combining results (1) reduces the gain (1) / more cores increase cost and power use (1) 3. Sequential part = 60 − 45 = 15 s (1) / on 3 cores: 15 + 45 ÷ 3 = 15 + 15 = 30 s (1) / shortest possible time = 15 s (the sequential part) (1)

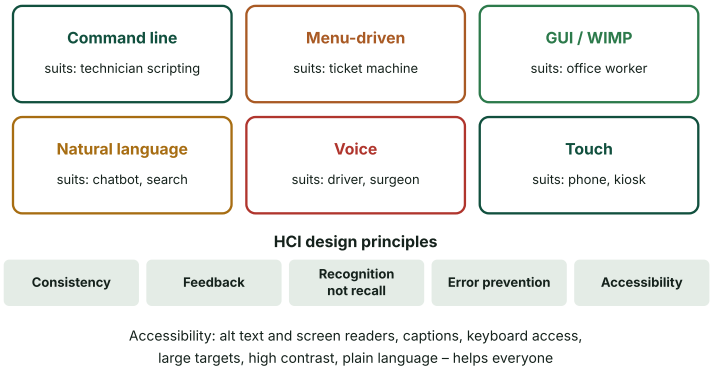
User interfaces and HCI design

FD12 FACT FILE

Specification: students should be able to

- Describe the features of a good user interface
- Describe the main features of WIMP (graphical), command line and forms dialogue interfaces
- Evaluate WIMP, command line and forms dialogue interfaces for given users and tasks
- Describe how text, sound and video can be input using personal computers, laptops, tablets and smartphones
- Describe other interfaces (voice, touch, biometric, menu-driven) and principles of HCI design, including accessibility

Key words: user interface, human–computer interaction (HCI), graphical user interface (GUI), WIMP, command line interface (CLI), switch (parameter), forms dialogue interface, menu-driven interface, voice (dialogue) interface, touch-sensitive interface, biometric interface, consistency, feedback, forgiving (undo), accessibility, speech recognition



Six types of user interface with an example user for each, and the key HCI design principles

What a user interface is and what makes it good

The user interface is the part of a system through which a person and a computer communicate: the screens, sounds, controls and input methods that let the user give instructions and receive results. A good interface is simple, with a clear layout that is easy to learn, striking a balance between offering many functions and staying uncluttered. It is intuitive and consistent, so controls work as users expect and in the same way throughout, allowing knowledge from one task to be reused in another. It is responsive, keeping up with the user's input and always giving feedback about what it is doing, such as a progress bar while a file uploads. It is forgiving, so mistakes can be reversed with undo or cancel, destructive actions are confirmed, and help is available. Finally, it is attractive, because good typography and colour make the system pleasant to use. These qualities underpin human–computer interaction (HCI) design, whose aim is to make systems effective, efficient and satisfying for the people who actually use them.

WIMP graphical user interfaces

A graphical user interface is the most common type of interface. It is often called a WIMP interface because it is built from windows, which are areas of the screen where applications and documents run; icons, which are small pictures representing programs, files and devices; menus, which list options to choose from; and a pointer, moved by a mouse, trackpad or finger, which selects items. Operating systems such as Windows, macOS, Android and iOS use graphical interfaces, usually alongside touch input. The main advantages are that a GUI is intuitive, because users recognise options instead of recalling commands, needs little training, makes it easy to exchange data between applications by copying and pasting, and offers help for beginners and keyboard shortcuts for experts. The disadvantages are that a GUI needs more memory, storage and processing power than other interfaces and a good-quality display, and it takes programmers longer to develop. Repetitive tasks can also be slower with a mouse than with a typed command or script.

Command line interfaces

A command line interface lets the user type commands at a prompt. The operating system recognises a fixed set of commands, each with a precise syntax, and many commands accept switches or parameters that change their behaviour, for example a switch that makes a copy or listing command include every subfolder. Command line interfaces were once used by everyone, but today they are mainly used by network managers, system administrators and programmers. Their advantages are that they need very little processing power, memory or display quality, and that an expert who knows the commands can work faster than with any other interface. Sequences of commands can be saved as scripts so that repetitive jobs, such as setting up dozens of user accounts, are automated. The disadvantages are that commands must be learned and typed exactly; a small mistake may mean the command fails or, worse, deletes files. There are few clues on screen about what to type, so the interface is confusing and unsuitable for novices.

Forms dialogue and other interfaces

A forms dialogue interface guides the user through a structured form using text boxes, drop-down lists, check boxes and radio buttons. It is widely used online for bookings, orders and applications. It is easy for beginners because everything is laid out logically and there are no commands to learn, it needs little processing power, and data can be validated as it is entered, reducing errors. However, a badly designed form can be slow, users can only choose from the options provided, and experienced users may find it tedious. A menu-driven interface offers a limited list of choices, often in levels, and suits kiosks and simple devices. Voice, or dialogue, interfaces let users speak commands; they are hands-free and help people who find typing difficult, but background noise, accents and words that sound alike cause errors. Touch-sensitive interfaces are intuitive and save space but make small targets hard to hit. Biometric interfaces use fingerprints or faces; they cannot be forgotten but are costly and raise privacy concerns.

Inputting text, sound and video

Different devices give users different ways of entering text, sound and video. A desktop PC usually has a full-size keyboard for fast, accurate text entry, and a scanner with optical character recognition can turn printed pages into editable text. Sound is captured through a plug-in or headset microphone, and video through a USB webcam or a capture card connected to a camera. A laptop builds these into the case: a keyboard and trackpad, a microphone array and a webcam above the screen, which makes video calls easy but gives lower quality than dedicated equipment. Tablets and smartphones rely on touch: text is entered with an on-screen keyboard, often with predictive text, swipe typing or handwriting recognition with a stylus. Speech recognition converts spoken words to text, which is quicker than typing on a small screen and helps users with dyslexia or limited mobility. Built-in microphones record voice notes, and front and rear cameras record high-resolution video that is compressed on the device so that it can be stored and shared.

HCI principles and accessibility

Human–computer interaction design applies a small number of principles. Consistency means the same layout, colours, terms and controls are used throughout, so users can predict how things work. Feedback means every action is acknowledged with a message, sound or visual change. Minimising memory load means showing options so that users recognise rather than recall them. Error prevention and recovery means validating input, confirming destructive actions and offering undo. The interface must also suit its users, tasks, environment and device. Accessibility means designing so that people with visual, hearing, motor or cognitive impairments can use the system. Examples include support for screen readers and alternative text for images, captions for video, keyboard-only navigation and large targets for people with limited dexterity, adjustable text size and strong colour contrast, never using colour as the only way of conveying meaning, and plain language with enough time to complete tasks. Accessible design helps everyone, including older users and people using a phone in bright sunlight, and supports legal duties under disability discrimination law.

T Key terms from the mark schemes

User interface
The means by which a user and a computer communicate, including what is displayed and how the user gives input.

Graphical user interface (GUI)
A user interface that uses windows, icons, menus and a pointer (WIMP) to allow the user to interact visually.
`Credited: WIMP interface`

Command line interface (CLI)
A user interface in which the user types commands in an exact syntax at a prompt for the computer to carry out.

Menu-driven interface
A user interface that presents the user with a limited list of options to choose from, often in a hierarchy.

Biometric interface
A user interface operated by measuring a physical characteristic of the user, such as a fingerprint, face or iris.

Human–computer interaction (HCI)
The study and design of how people interact with computers, aiming to make systems effective, efficient and satisfying to use.

Switch
An extra parameter typed after a command in a command line interface to modify what the command does.
Credited: Parameter / option

Forms dialogue interface
A user interface that guides the user through a structured on-screen form using text boxes, drop-down lists, check boxes and radio buttons.
Credited: Forms interface

Voice (dialogue) interface
A user interface that is controlled by spoken commands, using speech recognition.
Credited: Speech interface

Natural language interface
A user interface that accepts requests typed or spoken in ordinary everyday language.

Accessibility
Designing systems so they can be used by people with a wide range of abilities, including visual, hearing, motor and cognitive impairments.

C C# example

A tiny command interpreter in C#

```
string[] files = { "timetable.docx", "rota.xlsx" };
Console.Write("> ");
string command = Console.ReadLine() ?? "";
switch (command)
{
    case "list":
        foreach (string f in files) Console.WriteLine(f);
        break;
    case "count":
        Console.WriteLine(files.Length + " files");
        break;
    default:
        Console.WriteLine("'" + command + "' is not recognised");
        break;
}
```

Exam focus

Questions on interfaces are usually State, Describe or Explain items worth one or two marks per point, plus scenario questions asking you to recommend or evaluate an interface. Mark schemes credit the specific advantages and disadvantages listed for each interface: for example, a command line uses little memory and is fast for experts who know the commands, but commands must be typed precisely and it is unsuitable for novices. When asked why an expert might prefer a command line, link speed to their knowledge of the commands. For WIMP, name all four elements correctly: windows, icons, menus and pointer. For a forms interface, remember that validation can be carried out as data is entered. In recommendation questions, name the interface and justify it using the user's characteristics, such as expertise, disability, environment or task; vague phrases like 'easy to use' earn little unless you explain why and for whom. Extended questions need both benefits and drawbacks before a justified conclusion. Do not confuse a menu-driven interface with the menus of a GUI.

? Check yourself

1. A council is designing a self-service kiosk for paying parking fines. State four qualities its user interface should have. [4]
2. State what each letter of WIMP stands for. [2]
3. Give two advantages of a command line interface for an experienced network technician at a further education college. [2]

Answers: 1. Any four from: simple / clear, easy to learn (1) / intuitive and consistent (1) / responsive, giving feedback on status (1) / forgiving – mistakes easily reversed, help available (1) / attractive layout, typography and colour (1) / accessible (1) 2. Windows / icons / menus / pointer: all four correct (2); two or three correct (1). 3. Any two from: fast to use when the commands are known (1) / commands can be scripted to automate repetitive tasks (1) / needs little processing power or memory, so works on low-specification or remote devices (1) / gives precise, powerful control via switches/parameters (1)

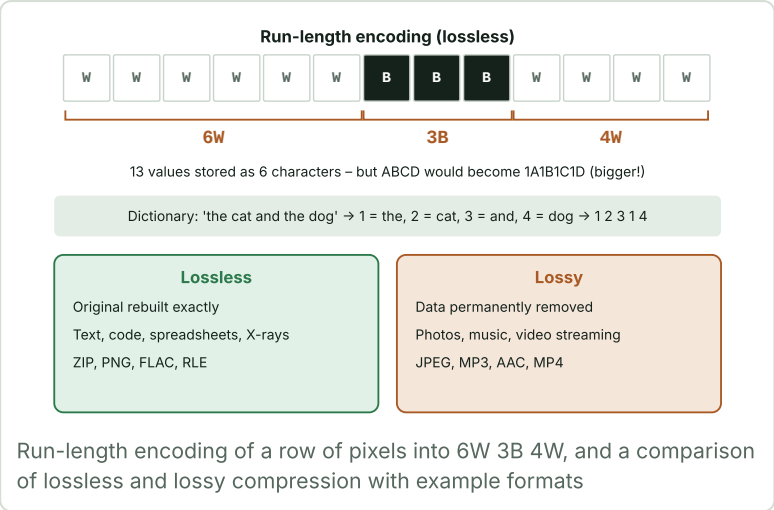
Data compression and file formats

FD13
FACT FILE

Specification: students should be able to

- Explain the need for data compression
- Distinguish between lossy and lossless compression and give appropriate uses of each
- Describe run-length encoding (RLE) and dictionary-based compression and apply them to simple data
- Describe how zipping is used to compress data
- Evaluate common data file formats: txt, wav, bitmap, JPEG, MPEG and GIF
- Calculate uncompressed file sizes and compression ratios

Key words: compression, redundancy, compression ratio, lossless, lossy, run-length encoding (RLE), dictionary-based compression, token, zipping, TXT, WAV, bitmap (BMP), colour depth, resolution (dpi), GIF, JPEG, MPEG (MP4), palette



Run-length encoding of a row of pixels into 6W 3B 4W, and a comparison of lossless and lossy compression with example formats

Why compress data?

Data compression means encoding data using fewer bits than the original representation. It works because most files contain redundancy: the same words, colours or sounds are repeated, and repetition can be stored more efficiently. There are two main reasons for compressing data. First, smaller files take up less storage space on phones, computers and servers. Second, smaller files are quicker to upload, download and stream and use less network bandwidth, which allows web pages to load quickly and video to be watched on slower connections. The cost is processing time to compress and decompress the data, and, with lossy methods, a permanent loss of quality. Images show the need clearly: a six by four inch photograph scanned at 300 dots per inch with 24-bit colour is 1800 by 1200 pixels, which at three bytes per pixel is 6 480 000 bytes, about 6.2 MB. A web page with twenty such images would be very slow to load. The compression ratio, the original size divided by the compressed size, measures how effective compression has been.

Lossy and lossless compression

Lossless compression reduces file size by removing redundancy without discarding any data, so decompression restores the original exactly. It is essential for text, program code, spreadsheets, financial records and medical images, where one wrong character or pixel could matter. Lossy compression permanently removes data that is judged to be less noticeable to people. In JPEG images, fine detail and small colour differences are reduced, because the eye is more sensitive to changes in brightness than in colour. In MP3 audio, frequencies outside the range of human hearing, and quiet sounds masked by louder ones, are removed. In MPEG video, much of each frame is predicted from the frames around it, so only the changes are stored. Lossy methods give much smaller files, often a tenth of the original size or less, and are suitable for photographs, music and video where a perfect copy is not needed. However, the original can never be recovered, higher compression means visibly lower quality, and saving a lossy file repeatedly makes the quality worse each time.

Run-length encoding

Run-length encoding is a simple lossless method that replaces each run of identical values with a count and a single copy of the value. A row of a black-and-white icon containing eight black pixels, sixteen white pixels and eight black pixels can be written as 8B16W8B, seven characters instead of thirty-two, a compression ratio of about 4.6 to 1. To decompress, the program reads each count and writes out the value that many times, restoring the original exactly. RLE works well on images with large areas of a single colour, such as icons, cartoons, logos and scanned documents with white backgrounds. It works badly on data with few repeats, such as photographs or ordinary text, because each single value needs a count as well: ABCD becomes 1A1B1C1D, which is twice as long. In practice, file formats use RLE alongside other techniques or only for runs above a minimum length. When answering questions, show the encoded output, count both sizes carefully and state the ratio or saving.

Dictionary-based compression and zipping

Dictionary-based compression is a lossless method that builds a table of repeated words or sequences and replaces each occurrence in the data with a short token that refers to the table. In the sentence 'the red bus and the blue bus and the green bus', six different words are stored once in a dictionary taking 21 characters, and the sentence becomes eleven one-byte tokens, so 46 bytes become about 32. The saving grows with longer files because common words appear many times while being stored only once. Real compression programs use more advanced versions, such as LZ77 and LZW, which build the dictionary automatically as they read the data, and these are found in ZIP, GIF and PNG. Zipping is the everyday use of this idea: one or more files are compressed losslessly into a single archive with a .zip extension, which is convenient for email attachments, backups and archiving. Any file can be zipped, but formats that are already compressed, such as JPEG or MP4, shrink very little.

Text, sound and image file formats

A plain text file (.txt) stores characters with no formatting, so it can be opened by almost any program on any platform, which makes it useful for exchanging data. A WAV file stores sound as uncompressed digital samples; at CD quality, 44 100 samples per second at 16 bits in stereo, one minute takes about 10.1 MB (10 584 000 bytes), giving excellent quality for recording and editing but large files. MP3 uses lossy compression to reduce this to about 1 MB per minute at 128 kbps. A bitmap (.bmp) divides an image into a grid of pixels and stores each pixel's colour as a binary number; the number of bits per pixel is the colour depth, and 24-bit colour gives 2 to the power 24, or 16 777 216, colours. Bitmaps are uncompressed and very large. When a bitmap is enlarged it becomes pixelated, which is why logos are often drawn as vector graphics instead. File size in bytes is width in pixels multiplied by height in pixels multiplied by colour depth, divided by eight.

GIF, JPEG and MPEG

GIF uses only eight bits per pixel, but each value is an index into a palette of up to 256 colours chosen from the full 24-bit range to suit that particular image. The pixel data is then compressed losslessly. GIF is good for logos, clipart, diagrams and simple animations, where each frame can have its own palette, but it is poor for photographs, which need thousands of shades. JPEG keeps 24-bit colour and uses lossy compression, so it is ideal for photographs with smooth changes of shade; the user can choose the level of compression, trading file size against visible quality, and heavily compressed JPEGs show blocky artefacts. MPEG is a family of standards for compressed video and audio; MPEG-4, usually stored as .mp4, carries both in a single stream with everything needed for playback. Encoding MPEG video takes much more processing than decoding it, so it suits content that is created once and watched many times, such as television catch-up services and online video sites. Choosing a format means balancing quality, file size, compatibility and purpose.

Key terms from the mark schemes

Data compression
Encoding data using fewer bits than the original representation, to reduce storage space and transmission time.

Lossless compression
Compression that reduces file size without losing any data, so the original can be reconstructed exactly.

Lossy compression
Compression that permanently removes data judged less important, giving smaller files that cannot be restored exactly.

Run-length encoding (RLE)
A lossless compression method that replaces a run of identical values with a single value and a count.

Dictionary-based compression
A lossless compression method that stores repeated patterns once in a dictionary and replaces each occurrence with a short reference (token).

Zipping
A lossless method of compressing one or more files into a single archive file with a .zip extension.

Bitmap (BMP)
An uncompressed image format that stores the colour of every pixel in a grid as a binary number.

JPEG
An image format that uses lossy compression on 24-bit colour images, suited to photographs.
Credited: Joint Photographic Experts Group

Compression ratio
The original file size divided by the compressed file size.

Colour depth
The number of bits used to represent the colour of each pixel; n bits give 2^n possible colours, and more bits give a larger file.
Credited: Bit depth

GIF
An image format using up to 256 colours from a palette chosen for the image, compressed losslessly; supports animation.
Credited: Graphics Interchange Format

MPEG
A family of standards for storing compressed video and audio, using lossy compression; MPEG-4 is usually stored as .mp4.
Credited: Moving Picture Experts Group

C C# example

Run-length encoding (RLE)

```
string data = "WWWWWWBBBWWWW";
string result = "";
int i = 0;
while (i < data.Length)
{
    int count = 1;
    while (i + count < data.Length && data[i + count] == data[i]) count++;
    result = result + count + data[i];
    i = i + count;
}
Console.WriteLine(result);   // 6W3B4W
```

Exam focus

Short questions ask for the two reasons for compression: reducing storage space and reducing transmission time. Be precise about the methods: lossy compression permanently removes data, while lossless compression allows the original to be reconstructed exactly. When asked what difference lossless would have made, say there would be no loss of quality but a larger file. Calculation questions usually ask for a bitmap file size; show the pixels as width in inches times dpi multiplied by height in inches times dpi, multiply by the bytes per pixel and state the units, remembering that 24-bit colour is three bytes and gives 16 777 216 colours. Increasing the colour depth increases the file size proportionally. For RLE or dictionary questions show the encoded output and both sizes. When asked why zipping is better than lossy compression for sending a logo to a printer, explain that zipping is lossless, so the printed logo keeps full quality. For file formats, link each one to suitable content: GIF for few colours, JPEG for photographs. A common error is calling RLE or GIF lossy.

? Check yourself

1. State two reasons for compressing files before uploading them to a school website. [2]
2. Explain the difference between lossy and lossless compression. [2]
3. A bitmap image measures 3 inches by 4 inches, has a resolution of 150 dpi and a colour depth of 24 bits. Calculate the file size in bytes. Show your working. [3]

Answers: 1. To reduce the file size so it takes less storage space on the web server (1) / to reduce upload/download (transmission) time so pages load faster (1) 2. Lossy permanently removes some data, so the original cannot be restored exactly (1) / lossless removes redundancy without losing any data, so the original can be reconstructed exactly (1) 3. Pixels: (3 × 150) × (4 × 150) = 450 × 600 = 270 000 pixels (1) / 24 bits = 3 bytes per pixel (1) / 270 000 × 3 = 810 000 bytes (≈ 791 KB) (1)

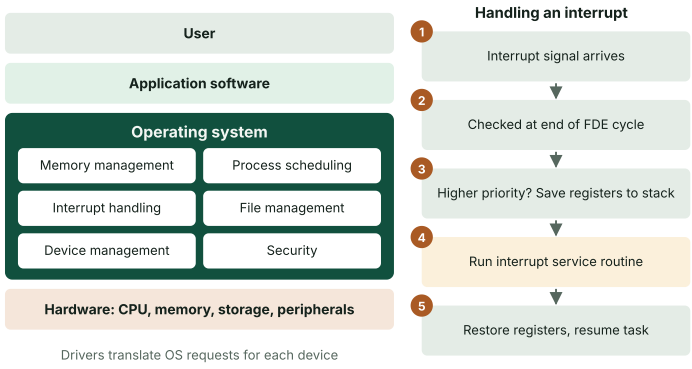
Operating systems and utility programs

FD14 FACT FILE

Specification: students should be able to

- Describe the purpose of an operating system and its main components: user interface, application interface, kernel and hardware interface
- Describe different types of operating system: single user, multi-user, multiprogramming, multiprocessing, multitasking and multithreading
- Explain memory management (paging and segmentation), process scheduling and the handling of interrupts
- Describe file management, device management using drivers, and the security functions of an operating system
- Explain the need for utility programs
- Describe common utility programs: data compression, file backup, archive software and disk defragmenters

Key words: operating system (OS), kernel, device driver, multi-user, time-slicing, multiprogramming, multitasking, multiprocessing, multithreading, deadlock, paging, segmentation, round robin, interrupt, interrupt service routine (ISR), utility program, file archiving, disk defragmenter



The operating system sits between applications and hardware and provides memory management, scheduling, interrupt handling, file and device management and security; an interrupt saves registers, runs the service routine and restores them

Overview: what an operating system is for

An operating system is the system software that manages every operation and resource of a computer. It sits between the hardware, the application software and the user, so a word processor can ask the OS to save a file rather than driving the disk itself. The OS is loaded first when the machine starts and stays in control while it runs. It can be pictured as four parts. The user interface takes input and presents information back, through a GUI or a command line. The application interface offers services that programs call and holds the device drivers that let the OS talk to each piece of hardware. The kernel is the core: it loads programs into memory, removes them when they finish, and shares memory, storage and processor time between them. The hardware interface links the CPU, memory and peripherals. Because an OS issues machine instructions, it must be built for a particular processor family. Alongside the OS sit utility programs, which together with the OS make up system software. Application software, by contrast, exists to help the user do a job, and always reaches the hardware through the OS.

Single user and multi-user systems

Operating systems can be classified by how many people and programs they support at once. A single user, single application OS lets one person run one program at a time and cannot distinguish between users; early desktop systems and simple embedded controllers worked like this. A single user, multitasking OS, the type found on most laptops, desktops and tablets, lets one person keep several programs open and switch between them, with the OS sharing resources among them. It needs more memory but is far more flexible. A multi-user OS allows many people to use one computer at the same time, each through their own terminal, and is typical of mainframes and large servers that may serve thousands of users concurrently. To make this work the OS shares processor time by time-slicing, giving each user's task a very short turn so that everyone appears to have exclusive use of the machine. It must also manage memory, storage and data requests so that every user gets enough, and it must keep each user's files private from all the others through accounts, passwords and access rights. Security and fair allocation are therefore the two responsibilities that most distinguish a multi-user OS.

Multiprogramming, multitasking, multiprocessing and multithreading

These four terms describe how an OS lets several programs share one computer. In multiprogramming, more than one program is resident in main memory, but only one executes at any instant. When the running program has to wait, typically for input or output, the OS switches the CPU to another program, so the processor is kept busy and the user sees an illusion of parallel running. Multitasking means essentially the same thing in a more general sense: tasks share the CPU and memory, and the OS allocates resources and switches between them, often using time-slicing. Multiprocessing is different because it uses two or more processors or cores, so work can genuinely run in parallel; its main benefit is speed. A system can be both multiprogramming and multiprocessing. Multithreading lets a single process run several threads, each a separate path of execution that shares the process's memory and code. A web server, for example, can handle many visitors using one copy of the program, with a thread per request. Threads are efficient, but if two threads update shared data at the same time results can be corrupted, and if each waits for a resource the other holds, both stop forever: a deadlock.

Memory management and scheduling

Memory management means allocating RAM to each program when it loads, keeping programs apart so one cannot overwrite another, and reclaiming memory when a program ends. When RAM is full the OS uses virtual memory, moving data that is not currently needed to secondary storage. Paging divides memory into equal, fixed-size pages; a page table maps each page of a program to any free frame in RAM, so a program need not be stored in one continuous block, although a part-filled final page wastes some space. Segmentation divides a program into variable-sized logical units such as a subroutine or a data structure, recorded in a segment table with a base address and length; this matches the program's structure but leaves awkward gaps between segments. Scheduling decides which process runs on the CPU and for how long. The aims are to keep the processor busy, maximise throughput, treat processes fairly and give interactive users quick responses. Round robin gives each process a fixed time slice in turn and sends unfinished processes to the back of the queue. First come first served, shortest job first and priority scheduling are alternatives; real-time systems rely on priorities so that urgent events are handled first.

Interrupts, devices, files and security

An interrupt is a signal telling the processor that something needs attention, such as a key being pressed, a printer running out of paper, a timer expiring or a hardware failure. Software can raise interrupts too, for example when a program tries to divide by zero. At the end of each fetch–decode–execute cycle the CPU checks for interrupts. If one is waiting and has a higher priority than the current task, the contents of the registers are saved on the stack, the correct interrupt service routine is run, and the saved values are then restored so the original task carries on as if nothing had happened. This is far more efficient than polling every device continually. Device management relies on device drivers: small programs, usually written by the hardware maker for each OS, that translate the OS's general requests into instructions a specific device understands. The OS also buffers and queues data, for example spooling print jobs. File management gives files names and folders, records where each is stored and controls who may read or change it. Security functions include user accounts, passwords, access rights, encryption, a firewall and logging, and they matter most on multi-user and networked systems.

Utility programs

Utility programs are system software tools that carry out specific housekeeping jobs to keep a computer efficient, secure and recoverable. They are needed because users and administrators must do these jobs regularly, and many are now built into the OS. A data compression utility encodes files using fewer bits so they take less storage and transfer more quickly; reducing size is a means to use resources better rather than a goal in itself. A backup utility makes exact copies of files or databases, usually on a schedule and ideally off-site, so that originals can be restored after hardware failure, accidental deletion or a ransomware attack. An archiving program combines many files into one archive file or a set of them, often compressing them, for long-term storage or transfer. It records metadata such as file names, lengths, timestamps and access control lists so each file can be rebuilt exactly, and an administrator decides what is archived, where, for how long and who can access it. A disk defragmenter rearranges the clusters of each file on a magnetic hard disk so they are contiguous, cutting the head movement needed to read them. It can take hours and should not be used on solid state drives.

T Key terms from the mark schemes

Operating system
System software that manages a computer's hardware and software resources and sits between the hardware, the application software and the user so that they can work together.
`Credited: Software that controls the hardware and provides a platform for applications`

Kernel
The core of the operating system, which loads and unloads programs in memory and allocates memory, storage and processor time.

Device driver
Software that allows the operating system to communicate with and control a specific hardware device.

Multi-user operating system
An operating system that lets many users use one computer at the same time, sharing its resources by time-slicing while keeping each user's data private.

Time-slicing
Sharing processor time by giving each task or user a short, fixed period of CPU time in turn.
`Credited: Time slice`

Multiprogramming
Holding several programs in main memory at once and switching the CPU between them, although only one executes at any instant.

Multiprocessing
Dividing work between two or more processors or cores so that it is executed truly in parallel.
`Credited: Parallel processing`

Multithreading
Running several threads, each a separate path of execution, within one process so they share its memory and code.

Paging
A memory management technique that divides memory into fixed-size pages, mapped to physical frames by a page table.

Segmentation
A memory management technique that divides a program into variable-sized logical segments such as modules and data structures.

Interrupt
A signal to the processor from hardware or software indicating that an event needs attention; the current task is suspended while an interrupt service routine runs.

Round robin
A scheduling algorithm in which each process is given a fixed time slice of the processor in turn.

Utility program
System software that performs a specific task to help manage, maintain or protect the computer, such as compression, backup or defragmentation.
Credited: Utility software

Disk defragmenter
A utility that rearranges the data on a magnetic hard disk so each file occupies contiguous clusters, reducing access time.

C C# example

Multithreading in C#

```
using System;
using System.Threading;

int counter = 0;
object key = new object();

Thread a = new Thread(AddMany);
Thread b = new Thread(AddMany);
a.Start(); b.Start();
a.Join(); b.Join();          // wait for both threads
Console.WriteLine(counter);  // 200000

void AddMany()
{
    for (int i = 0; i < 100000; i++)
    {
        lock (key) { counter++; }  // one thread at a time
    }
}
```

Exam focus

Short questions such as 'State the purpose of an operating system' earn a mark for managing hardware and software resources and another for acting as the interface between hardware, applications and the user. 'Describe' questions on OS types expect two or three precise features each: for a multi-user OS, mention many users at once, their own terminals, time-slicing, managing resources fairly and keeping users' data private. For the four 'multi' terms, avoid circular answers like 'multitasking does many tasks'. Say that in multiprogramming several programs are in memory but only one executes at a time, that multiprocessing uses more than one processor for true parallelism, and that multithreading runs several threads within one process sharing its resources. Interrupt answers need the sequence in order: checked at the end of the cycle, priority compared, registers saved to the stack, ISR run, registers restored. In utility questions, give the benefit as well as the action: a defragmenter makes clusters contiguous so files are read faster. Common mistakes are calling a device driver hardware, saying that deleting a file erases its data, and recommending defragmentation for an SSD. Round-robin traces should be set out as a table of time intervals.

? Check yourself

1. State two purposes of an operating system. [2]
2. Describe the role of the kernel in an operating system. [2]
3. A health trust's patient records system runs on a multi-user operating system used by hundreds of staff at the same time. Describe how the operating system makes sure that each user gets fair use of the system and that their data is kept private. [4]

Answers: 1. Any two from: manages the computer's hardware resources (e.g. memory, processor, devices) (1) / provides an interface between the user and the hardware (1) / provides a platform/environment for application software to run and access the hardware (1) / manages files and security (1) 2. Any two from: core of the OS (1) / loads programs into memory and unloads them when finished (1) / allocates resources such as memory and secondary storage (1) / schedules use of the processor (1) 3. Any four from: time-slicing shares processor time between users' tasks (1) / so each user seems to have the computer to themselves (1) / memory and storage are allocated so each user has sufficient resources (1) / data requests are queued/managed (1) / each user has a user account and password to authenticate (1) / access rights restrict each user to their own files/records (1) / one user's files are kept private from others (1)

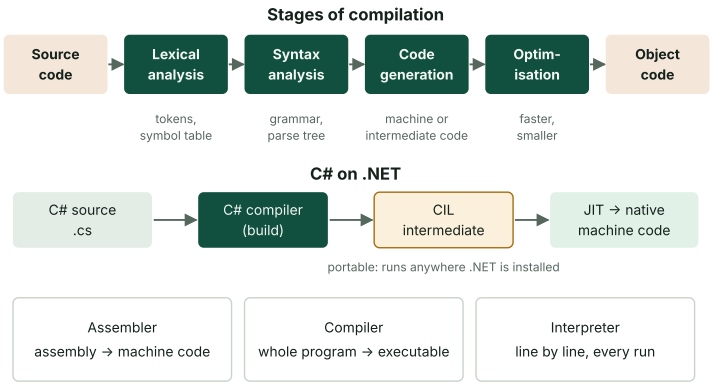
Translators, application software and processing systems

FD15 FACT FILE

Specification: students should be able to

- Describe the purpose of assemblers, compilers and interpreters, the stages of compilation and the use of intermediate code
- Explain what is meant by application software and the difference between generic and special purpose software
- Compare the use of custom-built software with off-the-shelf software
- Explain how application software can be delivered on different platforms, for example downloaded to a device or accessed using the web
- Evaluate different methods of obtaining software: proprietary and open source
- Describe batch processing and real-time processing and evaluate their use for common applications
- Describe the input, output and processing involved in ATMs, point of sale (POS) terminals and smartphones

Key words: compiler, interpreter, assembler, intermediate code, lexical analysis, application software, generic software, special purpose software, custom-built (bespoke) software, off-the-shelf software, platform, web application, proprietary software, open source software, batch processing, real-time processing, control total, EFTPOS



The stages of compilation from source code to object code, and how C# is compiled to Common Intermediate Language and then just-in-time compiled to machine code

Overview: from translators to processing systems

This lesson follows software from the moment it is written to the moment it is used. Programs written in high-level languages must be translated before a processor can run them, by a compiler, an interpreter, or a compiler that produces intermediate code for a virtual machine. The result is application software: programs that let users carry out their own tasks, such as writing a report or booking a ticket, and which reach the hardware only through the operating system. Application software can be generic, like a spreadsheet that serves countless purposes, or special purpose, built for one kind of job. An organisation can commission custom-built software or buy an off-the-shelf package, and it can install software on its own devices or use it through a web browser. It must also choose between proprietary software, owned and controlled by a company, and open source software, whose code anyone can examine and change. Finally, software processes data in different ways: batch processing collects transactions and handles them together later, while real-time processing deals with each one immediately. Everyday devices such as cash machines, shop tills and smartphones bring all of these ideas together as input, processing and output systems.

Translators and compilation

An assembler converts assembly language mnemonics into machine code, mostly one instruction for one instruction. A compiler translates a whole high-level program before it runs, producing an executable that runs quickly without the compiler and keeps the source code private; all the errors are listed together at the end. An interpreter translates and executes one statement at a time, stopping at the first error, which helps during development, but the program runs more slowly and needs the interpreter and the source code every time. Compilation passes through ordered stages. Lexical analysis removes comments and white space, splits the code into tokens and builds a symbol table of identifiers. Syntax analysis checks the tokens against the grammar of the language and builds a parse tree. Semantic checks confirm, for example, that variables are declared and types match. Code generation produces object code, and optimisation makes it faster or smaller. Languages such as Java and C# compile to intermediate code, bytecode or CIL, which is not tied to one processor. A virtual machine or runtime on each platform then interprets it or compiles it just in time into native code, combining portability with good speed.

Generic, special purpose, custom-built and off-the-shelf

Generic software can be used for many different tasks without being changed: the same word processor can produce a letter, an invoice or a set of instructions. Because it suits so many users it is written once and sold off-the-shelf in large numbers. Special purpose software is designed for one particular job or business need, such as a system that receives fault reports from machines and sends engineers to repair them. Some special purpose packages are sold to a whole sector, while others are custom-built, also called bespoke or tailor-made, either by in-house developers or by a software house hired by the client. Custom-built software does exactly what is needed, contains no unnecessary features and is supported by developers who know it well. However, it is expensive, takes a long time to deliver, may contain bugs that only show up under particular conditions, and there is little help available online. Off-the-shelf software is available immediately, costs less because development is shared among many buyers, has been tested by large numbers of users and comes with plenty of online support. On the other hand, it may need time to configure, may include features that are never used and may lack something the organisation requires.

Platforms and ways of delivering software

A platform is the base that software runs on: the hardware, particularly the type of processor, the operating system, or the combination of both. Windows and macOS have long run on similar Intel processors yet count as different platforms, because each provides a different application programming interface. Traditionally, a program written for one platform had to be rewritten for another. Cross-platform approaches reduce this effort. Java compiles to bytecode for a virtual processor, and a Java Virtual Machine on each platform runs that same bytecode. Web applications go further: most processing happens on a remote server and the user interface runs in a browser, so any device with a modern browser can use the application. Software reaches users in two main ways. It can be downloaded as an installation package that sets the program up on the device so it runs on local resources, works offline and is paid for by a one-off fee or by a subscription that disables it if not renewed. Alternatively it can be used through the web, running on the publisher's servers, needing little local power but a reliable internet connection, receiving updates centrally and usually charged by renewable subscription.

Proprietary and open source software

Proprietary software, sometimes called closed source, is owned by a person or company and protected by copyright. Users buy a licence to use it, but the source code is not released and the licence forbids copying or modifying it. In return, users usually receive regular updates, documentation and professional technical support, and the products tend to be polished and easy to use. The disadvantages are cost, dependence on the vendor for fixes and new features, paying for features that are not needed, and the risk that support ends if the company stops trading. Freeware is proprietary software given away at no charge; it is still closed source and its licence still has restrictions. Open source software is distributed with its source code under a licence that allows anyone to use, study, change and share it. Copyleft licences such as the GNU GPL also require that modified versions are distributed with their source code. Open source is usually free and not tied to one developer, so development can continue if the original authors stop. However, it may be less thoroughly tested or documented, support often comes from community forums, and several competing versions may exist. Linux and Android's underlying system are well-known examples.

Batch and real-time processing

In batch processing, transactions are gathered over a period into a transaction file and then processed together as one job at a scheduled time, often overnight when the system is quiet. The same operations are carried out on every record, and once the job has been set up it runs without anyone needing to intervene. Batch totals and control totals check that no data has been lost or added. Batch processing is efficient and cheap, and suits jobs where the result is only needed periodically, such as payroll, gas and electricity bills, monthly credit card statements and end-of-day stock reordering. Its weakness is that the master file is out of date until the batch has run. In real-time processing, each transaction updates the master file as soon as it occurs, so information is always current. Bookings and cash machines, where the user waits a few seconds, are real-time transaction (online) processing; systems reacting to sensors within a guaranteed time, such as traffic lights, are real-time control. Real-time processing is essential where a delay could cause harm or loss, such as preventing a concert seat being sold twice, stopping a customer withdrawing more than their balance, or sounding a heart monitor alarm. Because many requests can arrive together, real-time systems use scheduling algorithms to deal with the most urgent first. They need powerful, reliable, often duplicated hardware, so they cost more.

Devices as data processing systems

An automated teller machine is linked to a bank's computer system. The customer inserts a card into the card reader and types a PIN on the keypad; the PIN is sent, encrypted, to the card issuer's bank to be checked. After the menu choice and amount are entered, the system checks the balance, authorises the withdrawal and deducts the amount immediately. Output comes from the screen, the cash dispenser and the receipt printer. A point of sale terminal is a computerised till that also records stock and orders. Barcode scanners, touchscreens and electronic scales provide input; the system looks up each price in a product database, calculates totals and updates stock levels. With EFTPOS, the card reader and PIN pad send payment details to the customer's bank, which transfers the funds within seconds. Outputs include displays, receipts and warning beeps. A smartphone combines phone and computer features under a mobile operating system such as Android or iOS. Its inputs include the touchscreen, microphone, cameras, fingerprint sensor, motion sensors and GPS, plus data from Wi-Fi and mobile networks. Outputs include the screen, speakers, vibration and NFC for contactless payments.

T Key terms from the mark schemes

Compiler
A translator that converts a whole high-level program into object code before it is executed, producing an executable file.

Interpreter
A translator that translates and executes a high-level program one statement at a time, without producing an executable file.

Assembler
A translator that converts assembly language into machine code.

Intermediate code
Processor-independent code, such as Java bytecode or C# CIL, produced by a compiler and executed by a virtual machine or JIT compiler.
`Credited: Bytecode`

Lexical analysis
The first stage of compilation, in which comments and white space are removed, the source code is converted into tokens and a symbol table is built.

Application software
Programs designed to let the end user carry out specific tasks, such as word processing or booking tickets, accessing the hardware through the operating system.

Generic software
Application software that can be used for many different tasks without modification, such as a word processor or spreadsheet.
`Credited: General purpose software`

Special purpose software
Application software designed to carry out one specific task or meet a particular business need.

Custom-built software
Software written specifically for one organisation's requirements, by in-house developers or a third party.
`Credited: Bespoke / tailor-made software`

Off-the-shelf software
Ready-made software developed for a wide market and sold commercially to many customers.

Utility software
System software that helps to maintain, configure and protect a computer, such as antivirus or backup software.
`Credited: Utility program`

Open source software
Software distributed with its source code under a licence that allows users to view, modify and redistribute it.

Proprietary software
Software owned by an individual or company and distributed under a licence that keeps the source code secret and restricts copying and modification.
`Credited: Closed source software`

Batch processing
Processing in which transactions are collected over a period in a transaction file and processed together as one job at a later time, without user intervention.

Real-time processing
Processing in which each transaction or input is dealt with as soon as it occurs, so the master file is always up to date; real-time control systems also guarantee a response fast enough to affect the next input.

C C# example

`A batch run with a control total (C#)`

```
using System;

int[] stock = { 40, 25, 60 };            // master file
(int item, int qty)[] sales =             // transaction file
    { (0, 3), (2, 10), (0, 5), (1, 4) };
int controlTotal = 22;                    // counted at the tills

int sum = 0;
foreach (var s in sales) sum += s.qty;
if (sum != controlTotal)
{
    Console.WriteLine("Control total mismatch: batch rejected");
    return;
}
foreach (var s in sales) stock[s.item] -= s.qty;
for (int i = 0; i < stock.Length; i++)
    Console.WriteLine($"Item {i}: {stock[i]}");
```

Exam focus

Expect 'describe two advantages and two disadvantages' questions on custom-built and off-the-shelf software. Each point should be a full reason, for instance 'off-the-shelf software is cheaper because the development cost is shared among many buyers', not just 'cheaper'. When asked to recommend software in a scenario, use the details given, such as budget, deadlines or staff programming skills, and finish with a justified choice; a business owner with programming experience could adapt open source code, for example. Do not describe open source as simply 'free': the defining feature is that the source code is available and may be modified. Write 'proprietary', not 'propriety'. For batch processing, credit grouping transactions, processing at a set time without intervention, off-peak running and validation with control totals. For real-time, explain the consequence of delay, such as stock being sold twice or an account going overdrawn. Input, processing and output questions on ATMs, POS terminals and smartphones are marked per example, so give separate, specific items and put each in the right category: a card reader is an input device, checking the PIN is processing, and a receipt is output. Translator questions need paired comparisons, and compilation stages must be named in order.

? Check yourself

1. Describe two differences between a compiler and an interpreter. [4]
2. Describe what happens during the lexical analysis and syntax analysis stages of compilation. [4]
3. A dental practice in Bangor uses a spreadsheet for its accounts and a dedicated package for appointments and patient records. Using these two programs, explain the difference between generic and special purpose software. [4]

Answers: 1. Any two differences, 2 marks each: a compiler translates the whole program before it runs (1) whereas an interpreter translates and executes one line/statement at a time (1) / a compiler produces an executable/object code file (1) whereas an interpreter produces no executable so must translate every time it runs (1) / a compiler reports all errors after compilation (1) whereas an interpreter stops at the first error (1) / compiled programs run faster (1) because interpreted code is translated at run time (1) / compiled code can be distributed without the source code (1) whereas interpreted programs need the source code and interpreter (1) 2. Lexical analysis, any two from: white space and comments removed (1) / code split into tokens such as keywords, identifiers and operators (1) / symbol table of identifiers built (1). Syntax analysis, any two from: tokens checked against the grammar rules of the language (1) / parse/syntax tree built (1) / syntax errors reported (1) 3. Generic software can be used for many different tasks without modification (1), e.g. the spreadsheet could also be used for stock lists or rotas (1) / special purpose software is designed for one specific task or business need (1), e.g. the appointments package only manages dental bookings and patient records (1)

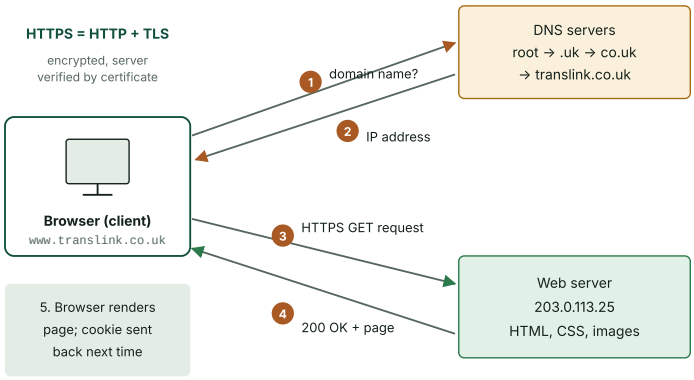
How the internet and the web work

FD16
FACT FILE

Specification: students should be able to

- Explain the terms Internet, World Wide Web (WWW), website, web page, web browser, web server, web hosting and internet service provider (ISP)
- Explain Internet Protocol (IP) addresses and how domain names are organised, including the role of the Domain Name System (DNS)
- Explain Uniform Resource Locators (URLs), HTTP, HTTPS and File Transfer Protocol (FTP)
- Explain the use of cookies, sessions and caches with the stateless HTTP protocol
- Explain how a search engine works
- Describe the role of the ITU and W3C (and the IETF) in agreeing web standards

Key words: Internet, World Wide Web (WWW), TCP/IP, IP address, IPv4 / IPv6, domain name, top-level domain (TLD), Domain Name System (DNS), internet service provider (ISP), uniform resource locator (URL), web browser, web server, HTTP / HTTPS, stateless protocol, cookie, session variable, cache, FTP, search engine



Loading a web page: the browser asks DNS for the IP address of the domain name, then sends an HTTPS request to the web server, which returns the page

Overview: the Internet and the Web

The Internet and the World Wide Web are often treated as the same thing, but they are different. The Internet is the global network of networks, linked by agreed protocols called TCP/IP so that any connected device can exchange data with any other. It grew from research networks in the 1960s and 1970s, including ARPANET in the United States, the NPL network in England, which introduced packet switching, and CYCLADES in France. Because these networks were incompatible, TCP/IP was developed as a common set of rules, and joining networks this way became known as internetworking. The World Wide Web is one of the services that runs over the Internet. Tim Berners-Lee proposed it at CERN in 1989 as a system of documents stored on servers and linked by hypertext, and CERN made it free for anyone to use in 1993. A web page is one hypertext document; a website is a group of related pages sharing a domain name and served by the same web server; a browser is the client program that requests and displays pages. Email, file transfer and online gaming also use the Internet, but they are not part of the Web.

IP addresses, domain names and DNS

Every device on the Internet must have a unique address, its IP address. A dynamic address is assigned each time a device connects, while a static address stays the same, which suits servers. IPv4 addresses are 32 bits long, giving roughly 4.3 billion possibilities, and are written as four 8-bit octets in decimal separated by full stops, such as 86.47.193.12. The growth of the Internet has used up IPv4 addresses, so IPv6 uses 128 bits, about 3.4 × 10^38 addresses, written as eight groups of four hexadecimal digits separated by colons. Because numbers are hard to remember, people use domain names instead. A domain name is read from right to left: the final label is the top-level domain, such as .uk or .com, and each label to the left is a sub-domain of the part after it, so www.belfastzoo.co.uk is a sub-domain of belfastzoo.co.uk, which is part of co.uk. The Domain Name System translates names into IP addresses. It is a distributed hierarchy of servers that pass queries along and exchange updates, so a name still leads to the right device when its address changes, and answers are cached to speed up later look-ups.

Connecting, hosting and URLs

Homes and businesses reach the Internet through an internet service provider, which connects them to a Point of Presence, its access point for a geographical area. A router, often combined with a modem, shares one connection between many devices. Many ISPs also offer email, domain name registration and web hosting. To publish a website, an organisation rents space from a web host, whose package normally includes a fixed amount of storage, a limit on traffic, email addresses, domain registration, database support, backups and traffic statistics. Pages are uploaded to the host using the File Transfer Protocol, which authenticates the user with a username and password and can move files of any type in either direction. Anonymous FTP, with no login, is used only to make files publicly downloadable. Each resource on the Web is identified by a uniform resource locator. A URL combines the protocol, the domain name of the server and the path to the file, for example https://www.translink.co.uk/timetables/metro.html, and may finish with a query string carrying data. In general, pages are uploaded to the server by FTP and then requested by visitors' browsers using HTTP or HTTPS.

HTTP, cookies and sessions

The HyperText Transfer Protocol is the set of rules a browser uses to request web pages and a server uses to reply. It is stateless: every request is a separate transaction with no memory of any earlier request. This makes the protocol simple to implement and efficient, the two main advantages, but it causes problems whenever a site needs context. An online bank, for instance, must know that the person asking to transfer money is the same person who logged in a moment ago. Two techniques solve this. A cookie is a small piece of text that the site asks the browser to store; the browser then sends it back with each request, so the server can recognise the user, remember preferences or keep track of a shopping basket. A session variable is stored on the server for the duration of the visit, and the browser holds only a session identifier, usually inside a cookie. Sessions are more secure because the sensitive data never sits on the user's device. Cookies also raise privacy concerns, since tracking cookies can follow users across sites. In the UK, the Privacy and Electronic Communications Regulations, together with the UK GDPR, require consent before non-essential cookies are stored.

HTTPS, caching and the page request

When a URL is entered, the browser first looks in its cache for the page and the server's IP address. If the address is not known, a DNS resolver finds it by querying the DNS hierarchy. The browser then connects to the web server and sends an HTTP GET request; for HTTPS a TLS connection is set up first, the server proves its identity with a digital certificate and all data is encrypted. The server returns a status code such as 200 OK or 404 Not Found together with the HTML. The browser then requests the style sheets, scripts and images that the page refers to and renders the result. Because HTTP sends every page in full, items common to many pages, such as logos and menus, would be downloaded repeatedly. Caching reduces this. A browser cache on the device stores items that user has already fetched, and a proxy cache on a shared server, for example in a school, stores content that many users request. Requests are checked against the cache before going to the web server, reducing traffic, waiting time and cost. Pages whose content changes rapidly, such as live sports scores, can tell browsers not to cache them, or only for a short time.

Search engines

In the Web's early years, lists of servers were kept by hand, but the number of sites soon made that impossible, and automatic search engines appeared from 1994. A modern search engine has three main parts. The web crawler, sometimes called a spider, starts from a set of seed pages, collects every link on them, visits each linked page in turn and adds any new links to its list, aiming to reach as much of the Web as possible and returning regularly to spot changes. The index stores what the crawler finds on every page, including its text, images, video and metadata, in a dictionary-like structure so that each word can be looked up to find the pages containing it. Metadata is data about the page, such as its title and description, that is not part of the visible content. The query engine is the part the user sees: it takes the search terms, looks them up in the index and ranks the matching pages so the most useful appear first. Google rose to dominance around 2000 with its PageRank algorithm, which scores a page more highly when many other pages, particularly highly ranked ones, link to it. Modern engines combine many other ranking signals as well.

Web standards bodies

The Internet and the Web work only because devices, networks and browsers made by thousands of organisations follow the same rules. Several standards bodies agree these rules. The Internet Engineering Task Force is an open, largely volunteer organisation that develops the core Internet protocols, including IPv4, IPv6, email and the Domain Name System. The International Telecommunication Union is a United Nations agency responsible for telecommunications. Its standardisation sector, ITU-T, specifies how data is carried and exchanged between devices: transmission over cables, radio and satellite links, the coding of digital video, sound and images, and the format of the digital certificates used to secure e-commerce. The World Wide Web Consortium, founded by Tim Berners-Lee, brings together member organisations to agree the standards for web content and web applications, such as HTML, CSS and accessibility guidelines. Its standards mean a web page should look and behave the same in any browser on any device. In short, the ITU deals mainly with how data is formatted and transmitted, while the W3C deals with how information to be displayed on web pages is specified. Without such agreement, browsers, servers and networks from different makers could not work together.

T Key terms from the mark schemes

Internet
A global network of interconnected computer networks that use TCP/IP so that any connected device can communicate with any other.

IP address
A unique numeric address assigned to a device on a network such as the Internet: 32 bits in IPv4 and 128 bits in IPv6.

Top-level domain (TLD)
The last part of a domain name, such as .uk, .ie or .com, which may indicate the country or type of organisation.

Client–server model
A model in which client devices send requests to a central server, which processes them and returns responses.

Internet service provider (ISP)
A company that provides users with a connection to the Internet, often with services such as email and web hosting.

Cookie
A small piece of text data sent by a website and stored by the browser, then returned with each request so the site can remember information about the user.

World Wide Web
A collection of linked web pages and resources, each with a unique address, accessed over the Internet using HTTP and browsers.
`Credited: WWW`

Domain Name System (DNS)
A distributed, hierarchical system of servers that translates domain names into IP addresses.

URL
Uniform resource locator: the full address of a resource on the Web, made up of the protocol, domain name and path or file name.
`Credited: Web address`

Web server
A computer running software that stores web pages and delivers them to browsers when requested using HTTP.

HTTPS
Hypertext Transfer Protocol Secure: HTTP sent over an encrypted TLS connection, with the server authenticated by a digital certificate.
`Credited: HTTP over SSL/TLS`

Session
Data about a user's visit stored on the web server and identified by a session ID held by the browser.
`Credited: Session variable`

Cache
Temporary storage of previously retrieved web content, in the browser or on a proxy server, checked before requesting content from the web server.

File Transfer Protocol (FTP)
A standard protocol for transferring files between computers, commonly used to upload website files to a web host.

Exam focus

Definition questions on terms such as web server, ISP, cache or cookie usually give one mark for what the thing is and one for what it does or why it is used, so include both. When asked how the Internet and Web differ, say the Internet is the hardware network of networks and the Web is a service of linked pages accessed over it. Sequence questions on loading a page reward the correct order: DNS look-up of the IP address, HTTP request to the web server, response and rendering. Say that DNS converts a domain name into an IP address, not the reverse. For HTTP versus FTP, link FTP to uploading files to a server and HTTP to a browser requesting pages. Questions on statelessness expect both advantages, simplicity and efficiency, and an explanation of how cookies or session variables keep track of a logged-in user. To explain sub-domains, read the name from right to left. Learn the three search engine components with what each does. For standards, link the W3C with web content languages such as HTML and CSS and the ITU with communications and data formats. Common errors are calling DNS 'Domain Name Server' when the system is meant, and claiming IPv4 addresses are 128 bits long.

? Check yourself

1. Explain the difference between the Internet and the World Wide Web. [2]
2. Explain the meaning of each of the terms web page, website and web server, and describe the relationship between them. [4]
3. The IPv4 address of a council's web server is written in binary as 11000011 00010110 00001010 01000001. Convert it to dotted decimal notation, showing your working. [3]

Answers: 1. The Internet is a global network of interconnected networks / the physical infrastructure and protocols (1) / the World Wide Web is a service running over the Internet: a collection of linked web pages accessed with a browser (1) 2. Web page: a single hypertext document viewed in a browser (1) / website: a set of related web pages with a common domain name (1) / web server: computer/software that stores web pages and delivers them when requested by a browser using HTTP (1) / relationship: a website is made up of web pages, stored on and delivered by a web server, and together all websites form the WWW (1) 3. 11000011 = 128 + 64 + 2 + 1 = 195 / 00010110 = 16 + 4 + 2 = 22 / 00001010 = 8 + 2 = 10 / 01000001 = 64 + 1 = 65 (1 for working shown for at least two octets, 1 for at least three octets correct) / final answer 195.22.10.65 (1)

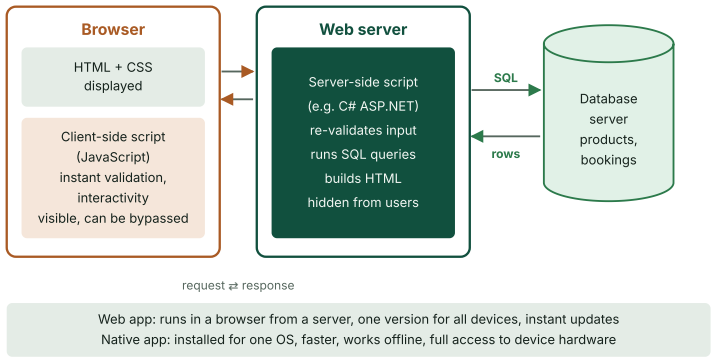
Web applications, intranets and web security

FD17 FACT FILE

Specification: students should be able to

- Compare client-side and server-side scripting and describe how dynamic web pages are generated using server-side scripts and databases
- Compare web applications with native applications
- Describe web-based services: e-services, e-government and portals
- Evaluate the use of an intranet by an organisation
- Evaluate methods of ensuring security over the internet: encryption (including public and private keys), HTTPS, Secure Sockets Layer (SSL/TLS), digital signatures and digital certificates

Key words: dynamic web page, client-side scripting, server-side scripting, SQL injection, web application, native application, e-service, e-government, portal, intranet, encryption, symmetric encryption, public key, private key, SSL/TLS, HTTPS, digital certificate, digital signature



A dynamic web page: client-side script runs in the browser, while the server-side script on the web server queries the database server and builds the HTML

Overview: applications and security on the Web

Many everyday tasks that once needed paper forms or a visit in person are now done through a web browser. Behind these services are web applications: programs that run partly in the user's browser and partly on a web server, and that build pages dynamically from information held in databases. Organisations use them to deliver e-services such as online shopping and banking, governments use them for e-government services such as licence renewals and tax returns, and portals collect many services together behind one consistent starting page. Inside an organisation, the same technology can form an intranet, a private web open only to staff. Because so much personal and financial data now travels across the public Internet, web applications must also be secure. Encryption makes intercepted data unreadable. Public and private key pairs allow strangers to communicate securely. TLS, the successor to SSL, protects the link between browser and server, and HTTPS is the result of carrying web traffic over it. Digital certificates prove that a public key belongs to the organisation it claims to, and digital signatures prove that a document is genuine and unaltered. This lesson covers how web applications are built and delivered, how organisations use them, and how they are kept secure.

Client-side and server-side scripting

A static page is a fixed HTML file that looks the same to every visitor until someone edits it. A dynamic page is generated when it is requested, usually by a server-side script that reads data from a database and builds HTML, so content can be personalised and kept current simply by changing the data. Client-side scripts, normally JavaScript, are downloaded with the page and run in the browser. They give an immediate response, for instance checking a form field as the user types or updating part of a page without reloading it, and they reduce the load on the server and the network. However, their code can be read, switched off or altered by the user, and their behaviour may vary between browsers. Server-side scripts, written in languages such as C# with ASP.NET, PHP or Python, run on the web server. They can query databases, process logins and payments and send emails, and because the code never leaves the server it remains hidden and secure. The cost is that every action needs a round trip to the server. Good design validates input on the client for usability and again on the server for security, using parameterised queries to prevent SQL injection attacks.

Web apps and native apps

A web application runs in a browser and is delivered from a server, so a single version works on any device and operating system with a modern browser. Nothing needs installing, updates take effect for every user the moment the server is changed, and the app can be shared simply by sending its URL. Its weaknesses are that it usually needs an internet connection, it may run more slowly, and it has more limited access to device hardware such as sensors, the file system or background notifications. A native application is written for a particular operating system, such as iOS or Android, and installed on the device. It runs faster, can use all of the device's features and works offline, and being listed in an app store helps people find it. However, separate versions must be developed and maintained for each platform, users must download updates, and app stores charge fees and impose approval rules. Progressive web apps narrow the gap by caching content for offline use and allowing installation to a home screen. The right choice depends on the purpose: a ticket booking service may suit a web app, while a fitness tracker that needs sensors and offline use suits a native app.

E-services, e-government and portals

E-services are services provided through electronic communication, usually the Internet. They include online shopping from internet-only firms and high street chains, online banking and insurance, live-chat helpdesks and digital editions of newspapers. For consumers the main benefit is convenience: services are available at any time from anywhere. Providers reach a wider customer base, gain an additional channel for contact with customers, can offer a larger range of products, and learn more about customers' preferences, which can give a competitive advantage; online-only retailers also avoid the cost of shop premises. E-government means governments providing their services online to the public, to businesses and to other bodies. It ranges from publishing information and guidance, through transactions such as vehicle tax or tax returns, to consultations and polls. It is generally faster and cheaper than paper-based systems, but governments must serve everyone, so people without equipment, connectivity or skills must not be disadvantaged. A portal is a website or page that gives consistent access to many connected services, often grouping links into rearrangeable blocks called portlets. An enterprise portal links an organisation's services for its staff or customers, usually after one log-in.

Evaluating an intranet

An intranet is a private network within an organisation, most often a private set of web pages and online services, that only authorised members can use. Its advantages are considerable. Policies, forms, training materials and staff directories can be published online, and because they are edited in one place only the latest version is available. Users sign in, so different groups can be shown only the links and documents relevant to them. Messaging, wikis, blogs and discussion boards improve communication and collaboration, online tools simplify tasks such as booking leave or scheduling meetings, and surveys help managers gauge and improve staff morale. There are also disadvantages. Setting up an intranet is expensive, and keeping its content accurate may require dedicated staff. Personal data on the intranet must be handled lawfully under the UK GDPR and the Data Protection Act 2018. A weak password could let an intruder reach confidential material, a risk that grows if staff can log in from outside the building. An organisation that relies heavily on its intranet can be badly disrupted if it goes down. A balanced evaluation weighs these points against the size and needs of the particular organisation and suggests ways of reducing the risks.

Encryption and public and private keys

Encryption converts readable plaintext into ciphertext using an algorithm and a key, so that anyone who intercepts the data cannot understand it. Only someone with the correct key can decrypt it. In symmetric encryption the same secret key is used to encrypt and to decrypt. This is fast and suits large amounts of data, but the sender and receiver must first share the key without anyone else learning it, which is difficult across the Internet. Public key, or asymmetric, encryption solves this by using a pair of mathematically linked keys, so that data encrypted with one key can only be decrypted with the other. The public key is published freely, while the private key is kept secret by its owner. To send someone a confidential message, you encrypt it with the recipient's public key; only the recipient's private key can decrypt it, so even if the message is intercepted it remains unreadable. Asymmetric encryption is much slower than symmetric encryption, so in practice the two are combined: public key techniques are used to agree a symmetric session key securely, and that session key then encrypts the actual data. This is exactly how TLS secures web traffic.

TLS, HTTPS, certificates and signatures

Secure Sockets Layer was the original protocol for creating an encrypted link between two devices; it has been replaced by Transport Layer Security, though the name SSL is still widely used. When a browser connects to a secure site, the two sides exchange greetings that list the encryption methods they support and include random values. The server sends its digital certificate, containing its public key. The browser checks the certificate, then the two agree a shared session key that an eavesdropper cannot work out, and all further data is encrypted with it. HTTP carried over this encrypted connection is HTTPS, shown by a padlock and using port 443. A digital certificate is issued and signed by a trusted certificate authority after checking the owner's identity. It contains the owner's name and domain, their public key, a serial number, the issuer and an expiry date, but never the private key. A digital signature serves a different purpose: it proves that a document came from a particular sender and has not been altered. The sender hashes the document and encrypts the hash with their private key. The receiver decrypts it with the sender's public key, recalculates the hash and compares the two values.

T Key terms from the mark schemes

Dynamic web page
A web page generated at the time of request, often from database data, so its content can change or be personalised.

Server-side scripting
Code that runs on the web server to process requests, access databases and generate pages before they are sent to the browser.

Native application
An application written for and installed on a specific operating system, with direct access to device hardware.
`Credited: Native app`

E-government
Government information and services provided electronically, usually online, to citizens, businesses and other organisations.

Intranet
A private network, usually of web pages and online services, within an organisation that only authorised members can access.

SSL/TLS
Protocols that create an authenticated, encrypted link between two devices, such as a browser and a web server; TLS is the modern successor to SSL.
`Credited: Secure Sockets Layer / Transport Layer Security`

Client-side scripting
Code, usually JavaScript, that is downloaded with a web page and run by the user's browser.

Web application
An application that runs in a web browser and is delivered from a server, processing user input and usually storing its data in a database on the server, without being installed on the device.
`Credited: Web app`

SQL injection
An attack in which SQL code is entered into an input field so that it is executed by the database.

Portal
A website or page that provides consistent access to a number of connected services from a single starting point.
`Credited: Enterprise portal`

Public key encryption
Encryption using a linked key pair: data encrypted with the freely available public key can only be decrypted with the owner's secret private key.
`Credited: Asymmetric encryption`

Digital certificate
An electronic document, issued and signed by a certificate authority, that links a public key to the identity of its owner.

Digital signature
A hash of a document encrypted with the sender's private key, used to verify who sent the document and that it has not been altered.

C C# example

Server-side script: building HTML from data (C#)

```
using System;
using System.Text;

(string Club, string Opponent, string Date)[] fixtures =
{
    ("Armagh", "Newry", "12 Apr"),
    ("Omagh", "Strabane", "19 Apr"),
    ("Armagh", "Lurgan", "26 Apr")
};
string club = "Armagh";              // from ?club=Armagh

var html = new StringBuilder("<ul>");
foreach (var f in fixtures)
    if (f.Club == club)
        html.Append($"<li>{f.Date}: v {f.Opponent}</li>");
html.Append("</ul>");
Console.WriteLine(html);
```

Exam focus

The specification asks you to evaluate the use of an intranet and to evaluate methods of ensuring security, so expect extended questions set in a business context. An evaluation needs advantages and disadvantages applied to the scenario and a justified conclusion, not two generic lists. For security questions, be precise about keys. A message for a recipient is encrypted with the recipient's public key and decrypted with their private key. A digital signature is created with the sender's private key and checked with the sender's public key. Mixing these up loses marks. Explain what each method achieves: encryption gives confidentiality, a signature gives authenticity and integrity, and a certificate links a public key to a verified owner. Do not say a certificate contains a private key. When describing HTTPS, mention both encryption of data in transit and authentication of the server by its certificate. For client-side and server-side comparisons, state where the code runs, who can see it, the effect on server load and response time, and why server-side validation is still needed. For web versus native apps, tie each point to the scenario, for example whether offline use or camera access is needed, and finish with a recommendation.

? Check yourself

1. Explain the difference between a static and a dynamic web page. [2]
2. Describe one advantage and one disadvantage of client-side scripting. [4]
3. Explain why form data should be validated on the server even if it has already been validated in the browser. [2]

Answers: 1. A static page is a fixed HTML file that is the same for every visitor until edited (1) / a dynamic page is generated when requested, e.g. by a server-side script using database data, so content can change or be personalised (1) 2. Advantage (2), one from: runs in the browser so gives an immediate response (1) e.g. instant validation feedback (1) / reduces load on the server/network (1) because fewer requests are sent (1). Disadvantage (2), one from: code is visible/can be altered or disabled by the user (1) so cannot be relied on for security/validation (1) / behaviour may differ between browsers (1) so must be tested on many (1) 3. Client-side validation can be bypassed, e.g. by disabling JavaScript or sending a request directly (1) / so the server must check data to protect the database, e.g. against SQL injection or invalid data (1)

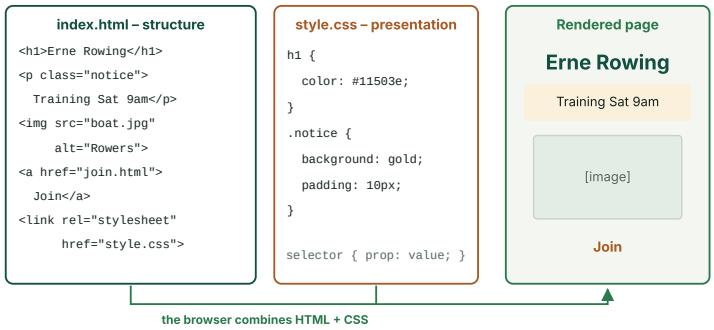
HTML and CSS

FD18
FACT FILE

Specification: students should be able to

- Explain how web pages are created using Hypertext Markup Language (HTML), including the html, head and body structure
- Describe the purpose of common HTML tags: paragraph, headings, line break, image, anchor (hyperlink), ordered list and unordered list
- Describe the purpose of tag attributes, empty and container tags, character entities and simple tables and forms
- Explain how Cascading Style Sheets (CSS) are used to control the presentation of web pages, using selectors, properties and values
- Compare inline, internal (global) and external style sheets and explain how inheritance and precedence resolve style conflicts
- Explain the benefits of separating content (HTML) from presentation (CSS)

Key words: HTML, tag, container tag, empty tag, attribute, hyperlink, anchor, alt text, character entity, CSS, selector, declaration, class, id, inline style, external style sheet, inheritance, precedence



One external style sheet styles every page: consistent, quick to change, cached by the browser

HTML provides structure and content, an external CSS style sheet provides presentation, and the browser combines them into the rendered page

How a web page is built with HTML

A website is a set of linked web pages, and each page is a text file written in Hypertext Markup Language. HTML does not decide how things look; it marks up the content so that a browser knows what each part is. Tags are keywords written inside angle brackets, and an element is usually an opening tag, some content and a closing tag in which a forward slash comes before the keyword. A modern page begins with the declaration <!DOCTYPE html>, followed by an html element containing two sections. The head holds information about the document as a whole: the title shown on the browser tab, the character set, and links to style sheets or scripts. The body holds everything the user actually sees, such as headings, paragraphs, lists, images, tables, forms and hyperlinks. When a user requests a page, the browser downloads the HTML, reads the tags from top to bottom, fetches any linked files such as images and style sheets, and renders the result in the window. Because HTML describes meaning rather than appearance, the same page can be displayed sensibly on a large monitor, a phone or a screen reader, with Cascading Style Sheets supplying the visual design.

Tags, attributes and text formatting

Most HTML elements use container tags, which come as a pair and wrap content, for example a paragraph or an anchor. A few use empty tags, which stand alone because they have no content to enclose; the line break, image, link and meta tags are common examples. Attributes refine an element and are written inside the opening tag as a name, an equals sign and a value in quotation marks, such as the href of a link or the src and alt of an image. Tag names are not case sensitive, but writing them consistently in lower case makes code easier to read. Browsers treat any run of spaces, tabs and new lines in the source as a single space, so pressing Enter in the code does not create a new line on screen; the br tag does that, while the p tag starts a new paragraph with a gap above it. Headings run from h1, the most important and largest, down to h6. Characters that are reserved in HTML, such as the less-than sign and the ampersand, or that are awkward to type, such as the copyright symbol or accented letters, are written as character entities beginning with an ampersand and ending with a semicolon.

Lists, hyperlinks, images and tables

An unordered list is enclosed in ul tags and shows a bullet before each item, while an ordered list uses ol tags and numbers its items; in both, each item sits inside li tags. The type attribute of an ordered list changes the numbering style, so a value of A gives capital letters and I gives Roman numerals. Hyperlinks are created with the anchor tag: the href attribute holds the destination and the text between the tags is what the user clicks. A relative address such as join.html points to another page on the same site, an absolute address beginning with https points anywhere on the web, and a mailto address opens a new email instead of a page. Images are added with the empty img tag. Its src attribute gives the file name, including the folder path if the image is stored elsewhere, and its alt attribute provides a text alternative that appears if the image fails to load and is read aloud by screen readers. Wrapping an img inside an anchor makes the picture clickable. Tables, which the specification does not require you to write but which are widely used, are built from table, caption, thead, tbody, tr, th and td elements.

Forms

Forms let a page collect data from users, for example a booking for a training session or a request for a brochure. The form element gives two important attributes: action names the server-side script that will receive the data, and method states how it is sent. With post, the data travels in the body of the HTTP request rather than being added to the address, which is more suitable for personal details. Inside the form, input elements have a type that controls both their appearance and what the browser accepts: text, email, number, password, date, checkbox, radio and submit are the most common. Each control should have a label, linked by matching the label's for attribute to the input's id, so that clicking the label selects the field and screen readers announce what it is for. HTML attributes provide simple validation in the browser before anything is sent: required acts as a presence check, min and max as a range check, maxlength as a length check, and pattern as a format check. These checks give quick feedback, but a user can bypass them, so the server must always validate the data again, as the next lesson explains.

CSS rules, selectors and colours

Cascading Style Sheets describe how HTML elements are presented. A CSS rule starts with a selector, which picks out the elements to be styled, followed by braces containing one or more declarations. Each declaration is a property, a colon and a value, and declarations are separated by semicolons. An element selector such as p applies to every paragraph. A class selector, written with a full stop, applies to every element given that class, so it suits styles that repeat, such as a highlighted fixture. An id selector, written with a hash, applies to the single element with that id, because an id must be unique within a page. Selectors can be grouped with commas or combined, so nav a means links that sit inside the navigation bar. Colours can be given in three ways: by one of the standard colour names, by rgb() with red, green and blue values from 0 to 255, or by a hash followed by six hexadecimal digits, two for each primary from 00 to FF. For example, green, rgb(0, 128, 0) and #008000 all give the same colour, because 128 in denary is 80 in hexadecimal. CSS uses the American spelling color, and background-color fills the area behind an element.

Inline, internal and external style sheets

Style information can be placed in three locations. An inline style is written in the style attribute of a single element; it is quick for a one-off change but has to be repeated on every element that needs it, which mixes presentation into the content and makes the site hard to update. An internal, or global, style sheet is a style block in the head of a page; its rules apply to every matching element on that page, but every other page needs its own copy. An external style sheet is a separate file with a .css extension, attached to each page with an empty link tag in the head that gives rel="stylesheet" and the file name in href. External style sheets are the most flexible approach. One file controls the house style of the whole site, so changing a colour scheme or font means editing one line rather than hundreds of pages, and every page stays consistent. The HTML files become smaller and the browser downloads and caches the style sheet once, so later pages load faster. Different style sheets can also be supplied for phones, printing or users who need high contrast, all using the same HTML content.

Resolving conflicts: inheritance and precedence

The word cascading refers to the predictable rules a browser uses when more than one style could apply to an element. Inheritance means that elements nested inside another element take on many of their parent's properties, especially text properties such as color, font-family and font-size; a span inside a blue Arial paragraph is also blue Arial unless told otherwise. Not every property is inherited: borders, padding and backgrounds belong to the element they are set on. Precedence means that when the same property is defined in more than one place, the definition closest to the element wins. An inline style overrides a rule for that element in an internal or external style sheet, and a rule set directly on a child overrides the value it would have inherited from its parent. A new property on the child simply adds to the inherited ones. For example, if paragraphs are blue Arial and a span inside one has an inline style of color: red, the span text appears in red Arial, because the font is inherited but the colour is overridden. If that inline rule were removed, the text would revert to blue. Where no rule applies at all, the browser's default style is used.

T Key terms from the mark schemes

HTML
Hypertext Markup Language: the markup language that uses tags to define the structure and content of a web page, such as headings, paragraphs, lists, images and links.
`Credited: Language used to create web pages`

Container tag
An HTML tag that is used as an opening and closing pair enclosing content, such as ... or <a> ... .
`Credited: Paired tag`

Empty tag
An HTML tag that stands alone with no closing tag because it encloses no content, such as
, <img> or <link>.
`Credited: Void element / self-closing tag`

Attribute
Extra information written inside an HTML opening tag as name="value", which refines the element, such as href, src or alt.

Anchor tag
The <a> tag, which creates a hyperlink; its href attribute gives the destination URL and the enclosed content is what the user clicks.
`Credited: Hyperlink tag`

Alt text
The alt attribute of an image, giving a text description that is displayed if the image cannot load and is read aloud by screen readers.
`Credited: Alternative text`

CSS
Cascading Style Sheets: the language used to describe the presentation of web pages, such as colours, fonts, spacing and layout, separately from the content.

Selector
The part of a CSS rule that identifies which HTML elements the declarations apply to, e.g. an element name, a .class or an #id.

External style sheet
A separate .css file linked to web pages with the <link> tag so that one set of style rules controls the appearance of the whole site.
`Credited: Linked style sheet`

Inline style
A style written in the style attribute of a single HTML element, affecting only that element.

Inheritance
The way a nested (child) element takes on style properties such as colour and font from the element that contains it (its parent).

Precedence
The rule that when the same property is defined in more than one place, the definition closest to the element (e.g. an inline style) takes priority.
`Credited: Closest definition wins`

Exam focus

Questions on this topic often ask you to write short pieces of HTML or CSS, to explain what code will display, or to compare ways of applying styles. When writing HTML, close every container tag, nest elements correctly, use the exact attribute names href, src and alt, and put attribute values in quotation marks; a mark is usually given for each correct element. Changing a bulleted list to a numbered one only needs the outer tags changed from ul to ol. In CSS, remember color without a u, a colon between property and value, semicolons between declarations, a full stop for a class and a hash for an id. When asked how text will appear, work through each property separately: say which value is inherited, which is overridden and why, using the words inheritance and precedence. Questions on the purpose of CSS should contrast it with HTML: HTML describes the content and structure, CSS describes the appearance. Questions on linked or external style sheets expect an explanation, not a list: one change in the CSS file is applied to every linked page, which keeps a house style consistent and reduces maintenance, and styles can be adapted for different devices.

? Check yourself

1. HTML elements are written using either container tags or empty tags. Describe what is meant by each type of tag and give one example of each. [4]
2. A walking club in the Mourne Mountains wants a page that shows the heading 'Upcoming walks', a sentence 'All walks start at 10am.', a numbered list of three walks (Slieve Donard, Slieve Binnian, Slieve Meelmore) and a link with the text 'Book a place' that opens booking.html. Write the HTML for the body of this page. [6]
3. Write the HTML to display the image logo.png, stored in a folder called images, so that clicking the image takes the user to index.html. Include suitable alternative text. [3]

Answers: 1. Container tag: comes as an opening and closing pair (1) which encloses content (1), e.g. ... , <a> ... , <h1> ... </h1> (1) / Empty tag: stands alone with no closing tag/no content, e.g.
, <img>, <link> (1). 2. <h1>Upcoming walks</h1> (1) / All walks start at 10am. (1) / ... used for the numbered list (1) / three <li> ... items with the correct text (1) / (1) ... Book a place with correct closing tag (1). Accept any heading level; accept ul only if numbering added by other means (not expected). 3. (1) / <img src="images/logo.png" (1) alt="[suitable description, e.g. Club logo – home page]"> (1) . The img must be inside the anchor for full marks.

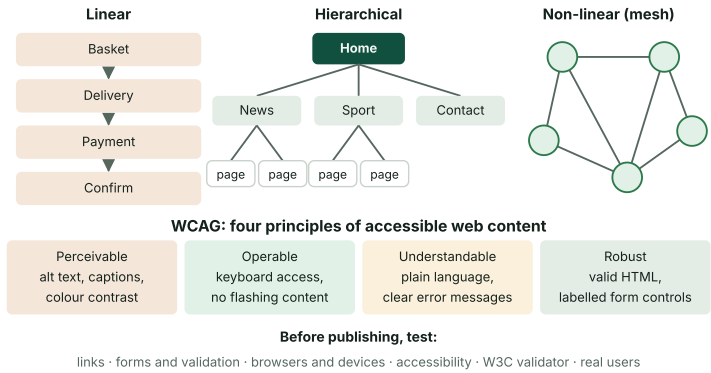
Website design, client-side and server-side processing, testing and publishing

FD19 FACT FILE

Specification: students should be able to

- Describe how a website is planned, including purpose, target audience, site maps and storyboards
- Describe linear, hierarchical and non-linear navigation structures and principles of usability
- Explain how websites can be made accessible, with reference to the Web Content Accessibility Guidelines (WCAG)
- Distinguish between static web pages and web applications, and between client-side and server-side processing
- Explain why form input should be validated both on the client and on the server
- Describe how a website is tested, published and maintained

Key words: target audience, site map, storyboard, wireframe, navigation structure, hierarchical structure, linear structure, usability, WCAG, responsive design, static web page, web application, client-side processing, server-side processing, validation, broken link, domain name, web hosting



Linear, hierarchical and non-linear navigation structures, and the four WCAG principles: perceivable, operable, understandable and robust

Planning a website

Building a good website starts long before any HTML is written. The developer first agrees the purpose of the site, such as informing parents about a school, selling tickets for a theatre or taking bookings for a leisure centre, and identifies the target audience: their age, interests, reading level, accessibility needs and the devices they are likely to use. These two decisions shape everything else, from the tone of the writing to the size of the buttons. Next the requirements are listed: the content needed, the functions the site must offer, such as a search box, a form or online payment, and any constraints on cost and time. A site map is then drawn, usually as a tree, showing every page and how the pages link, which makes sure no page is forgotten or unreachable and fixes the navigation structure. Storyboards or wireframes sketch the layout of each page, showing where the menu, headings, text, images and forms will appear, and annotations add details such as fonts, colours, link destinations and alt text. Agreeing a house style at this stage keeps every page consistent, and showing storyboards or a simple prototype to the client and to users finds problems while they are still cheap to fix.

Navigation and usability

The navigation structure describes how users move between pages. In a linear structure pages are visited in a fixed order using Next and Previous links, which suits processes such as a checkout or a multi-page application form where every step must be completed in turn. In a hierarchical structure the home page links to the main sections, each of which links to its own sub-pages; this is easy to understand and suits most organisational sites, although deep hierarchies need many clicks. A non-linear or mesh structure lets pages link freely to related pages, which suits wikis and news sites but can leave users lost. Most real sites combine these, for example browsing products hierarchically and then paying through a linear checkout. Usability is how easily the target audience can complete their tasks. Usable sites keep the menu in the same place on every page with a route back to the home page, keep key information within a few clicks, use clear headings, short paragraphs and readable fonts, and load quickly because images are compressed. Responsive design uses flexible layouts and CSS media queries so that the same content rearranges itself to fit a phone, tablet or desktop screen without sideways scrolling. Helpful, specific error messages also improve usability.

Accessibility and WCAG

An accessible website can be used by everyone, including people with visual, hearing, motor or cognitive impairments, and people using assistive technology such as screen readers, magnifiers or switch devices. The Web Content Accessibility Guidelines, published by the World Wide Web Consortium, are the international standard. The current version is WCAG 2.2, and its success criteria are graded at levels A, AA and AAA. They are organised under four principles. Perceivable means content can be seen or heard in some form, through alt text for images, captions and transcripts for audio and video, sufficient colour contrast and text that can be enlarged. Operable means every control can be used, including by keyboard alone, with no content that flashes in a way that could trigger seizures and with enough time to finish tasks. Understandable means plain language, predictable and consistent navigation, labelled form fields and error messages that explain how to fix the problem. Robust means valid, semantic HTML that assistive technologies can interpret reliably. In the UK, public sector bodies such as councils, health trusts and universities are legally required (schools are only partly covered) to meet WCAG level AA and publish an accessibility statement, and disability discrimination law requires other service providers to make reasonable adjustments, so accessibility is a legal as well as an ethical issue.

Static pages, web applications and client-side processing

A static web page is stored on the server as a finished file and is simply retrieved and displayed; it requires no calculation and no input from the user. A web application, by contrast, is a program accessed through a browser that processes what the user enters, such as an online banking service or a ticket booking system, and it usually keeps its state in a database so that the data changes as people use it. The processing in a web application is divided between the client and the server. Client-side processing is carried out by the browser on the user's own device, using HTML form attributes and JavaScript, without needing any further resources from the server. Its main job is to respond to the user quickly: checking that required fields have been completed, that values have the right type, length, format and range, that related fields have been completed together, and that a time limit has not expired, then displaying a clear message if anything is wrong. It can also change the page as the user works, for example updating a total or revealing extra questions. Only when all checks pass is the data sent, which avoids wasted requests, reduces load on the server and network, and gives the user instant feedback.

Server-side processing

Server-side processing happens on the web server after the browser has submitted a request. A script, written in a language such as PHP, C# with ASP.NET, Python or JavaScript running on Node.js, receives the data and identifies the transaction required, such as reserving two places on a class. It then fetches any data it needs from the database, for instance the number of places remaining, carries out the calculation, and writes the updated values back along with a record of the booking. If payment is involved, the server contacts a payment provider to authorise the transaction. Should any stage fail, for example a declined card or too few places left, the database is rolled back to its earlier state so that it remains consistent. Finally the server generates a response page, either a confirmation or an explanation of the problem, and sends it back for the browser to display. Server-side processing is essential for anything involving shared or confidential data, because the database and the program code remain on the server where users cannot see or alter them. It is also the only place where some checks can be made, such as whether a username already exists or whether a password matches the stored record.

Validating form input on both sides

Validation checks that input is sensible and in the expected form; it cannot prove that data is correct, only that it is reasonable. On the client, HTML attributes and scripts provide presence, type, length, range and format checks. In HTML, required acts as a presence check, min and max as a range check, maxlength as a length check and pattern as a format check, while input types such as email and number restrict what can be entered. These checks are fast and convenient, but they are under the user's control: JavaScript can be turned off, the page can be edited using the browser's developer tools, and a request can be sent to the server without using the form at all. For this reason the server must repeat every important check before storing or using the data. Server-side validation protects the database from invalid entries and from deliberately malicious input, such as code inserted into a text field in an attempt at SQL injection. The best practice is therefore to validate twice: on the client for speed and a better user experience, and on the server for security and integrity. Error messages should appear next to the field concerned and say exactly what is wrong and how to correct it.

Testing, publishing and maintenance

Before a website goes live it is tested systematically against a test plan that lists each test, the data used and the expected result. Functional testing follows every link to find broken links, which lead to missing pages and error messages such as 404 Not Found, and checks that forms accept valid data, reject invalid and extreme data with helpful messages, and submit correctly. Compatibility testing views the site in several browsers and on phones, tablets and desktops, because each may display pages slightly differently. Accessibility testing uses only a keyboard, a screen reader and a contrast checker, and code validation checks the HTML and CSS against the published standards. Performance testing measures loading times, including on a slow mobile connection, and usability testing asks real members of the target audience to complete tasks while observers note difficulties. To publish the site, the owner registers a domain name with a registrar, rents space from a web hosting company whose servers are permanently connected to the internet, uploads the files using FTP, SFTP or the host's control panel, points the domain's DNS records at the host and installs a certificate so the site is served securely over HTTPS. The live site is then tested again and maintained: content is updated, broken links are fixed, software is patched and backups are taken.

T Key terms from the mark schemes

Target audience
The group of people a website is designed for, described by factors such as age, interests, abilities, accessibility needs and the devices they use.

Storyboard
A sketch of the layout of each web page, showing the position of navigation, text, images and other elements, often with annotations.
`Credited: Wireframe`

Linear structure
A navigation structure in which pages are visited in a fixed sequence using next and previous links.
`Credited: Sequential structure`

Responsive design
Web design in which the layout adapts automatically to the size of the screen being used.

Client-side processing
Processing carried out by the browser on the user's device, such as validating form input or changing the page, without contacting the server.

Site map
A diagram showing all the pages of a website and how they are linked.
`Credited: Structure chart of the site`

Hierarchical structure
A navigation structure in which the home page links to main sections, which in turn link to their own sub-pages.
`Credited: Tree structure`

WCAG
Web Content Accessibility Guidelines: the international standard for accessible web content, based on the principles perceivable, operable, understandable and robust.

Web application
An application that runs in a web browser and is delivered from a server, processing user input and usually storing its data in a database on the server, without being installed on the device.
`Credited: Dynamic website`

Server-side processing
Processing carried out on the web server after a request is submitted, such as querying and updating a database and generating the response page.

Broken link
A hyperlink whose destination does not exist, producing an error such as 404 Not Found.
Credited: Dead link

Web hosting
A service that stores a website's files on servers permanently connected to the internet so the site can be accessed by users.

C C# example

Why validate on both client and server?

```
Console.Write("Postcode: ");
string pc = (Console.ReadLine() ?? "").Trim().ToUpper();
string[] parts = pc.Split(' ');
bool ok = parts.Length == 2 && pc.StartsWith("BT")
          && parts[0].Length >= 3 && parts[0].Length <= 4
          && parts[1].Length == 3 && char.IsDigit(parts[1][0])
          && char.IsLetter(parts[1][1]) && char.IsLetter(parts[1][2]);
for (int i = 2; ok && i < parts[0].Length; i++)
{
    if (!char.IsDigit(parts[0][i])) ok = false;
}
if (ok) Console.WriteLine("Format accepted - now check the database");
else Console.WriteLine("Enter a postcode such as BT74 6DQ");
```

Exam focus

Questions on client-side and server-side processing are often set in a scenario such as logging in to online banking or buying tickets, and the marks go to answers that apply both terms to that scenario. Say what the browser does, for example checking that both the username and password boxes are filled before anything is sent, and what the server does, such as retrieving the account details from the database, building the page and returning it to the browser. Saying only that one happens on the computer and the other on the server earns little. When explaining why validation is repeated on the server, state that client checks can be bypassed and that the server protects the data. Planning questions usually ask what a site map or storyboard shows and why it is useful, so give both parts. For navigation structures, choose one and justify it for the context given. Accessibility answers should name a specific feature and say whom it helps and how, for example that alt text is read aloud by a screen reader for a blind user. Test plan answers need specific tests with the data and expected outcome, not vague phrases like checking that it works. In extended questions, cover a range of points, link each to the context and finish with a justified conclusion.

? Check yourself

1. State two things a developer should find out about the target audience before designing a website. [2]
2. Describe what a site map shows and explain why one is produced before a website is built. [3]
3. A Belfast arts centre is designing its website. It has sections for shows, workshops and visiting, and a ticket-buying process. Describe a suitable navigation structure for the site, justifying your choice. [4]

Answers: 1. Any two from: age range (1) / interests or purpose of their visit (1) / reading level or IT skills (1) / accessibility needs, e.g. visual impairment (1) / the devices they use, e.g. mobile phones (1) / language (1). 2. Shows every page in the website (1) and how the pages are linked / the navigation structure (1) / produced first so no page is missing or unreachable and the navigation can be agreed before coding begins (1). 3. Hierarchical/mixed structure identified (1) / home page links to main sections: shows, workshops, visiting (1) / each section links to its own sub-pages, e.g. individual shows (1) / ticket buying uses a linear sequence (select seats, details, payment, confirmation) because steps must be completed in order (1). Justification must relate to the context.